



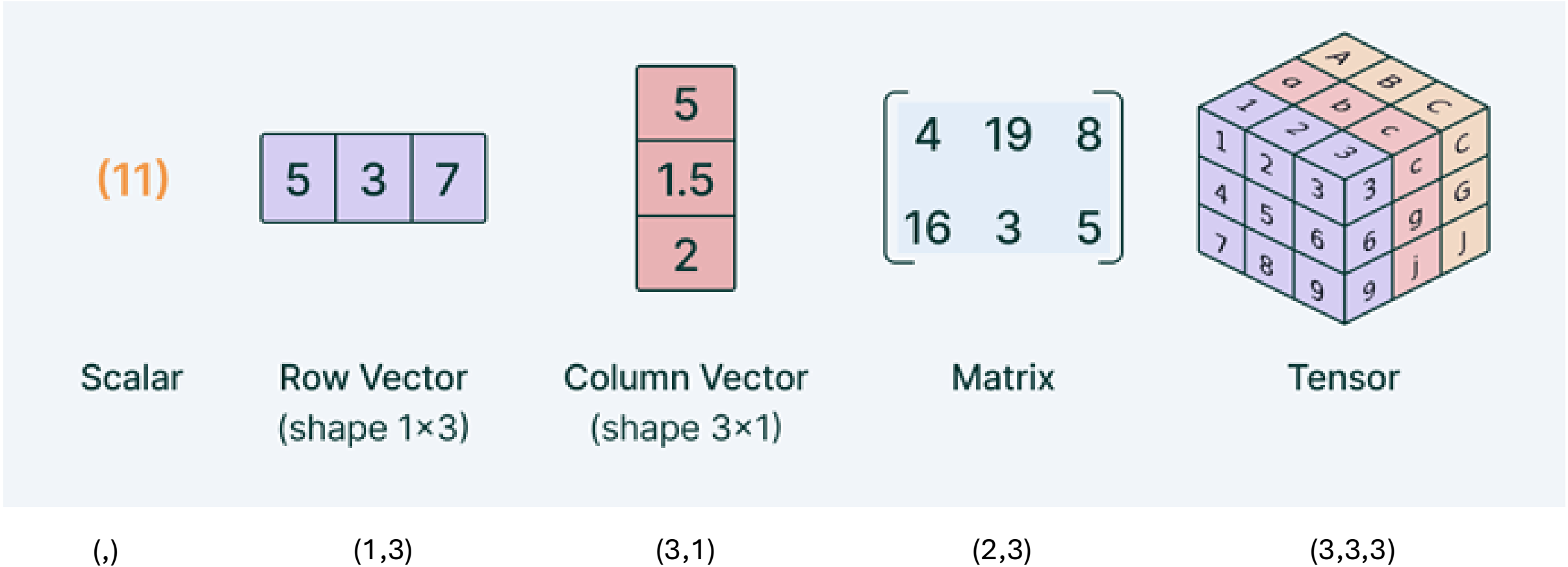
Applied AI Mastery Program

Module 1 – From Linear Models to Deep Learning

Rina Buoy, PhD



Recap – Scalar, Vector, Matrix, Tensor



Physical Properties as a Scalar or Vector

Scalar Quantities

length, area, volume
speed
mass, density
pressure
temperature
energy, entropy
work, power



Vector Quantities

displacement, direction
velocity
acceleration
momentum
force
lift, drag, thrust
weight



Linear Equations

Matrix multiplication

$$y = \mathbf{W}\mathbf{X}^T + w_0$$

(,)

(1,m) (m,1)

(,)



(1,1)=(,)

$$\mathbf{W} = [[w_1, \dots, w_m]]$$

$$\mathbf{X} = [[x_1, \dots, x_m]]$$

Linear Equations

$$y = [w_1, \dots, w_m] \begin{matrix} (1,m) & (m,1) \\ \left[\begin{array}{c} x_1 \\ \dots \\ x_m \end{array} \right] \end{matrix} + w_0$$
$$= \sum_{i=1}^m w_i x_i + w_0$$

Linear Equations

$$y = w_1x_{qa} + w_2x_{at} + w_3x_{hw} + w_4x_{te} + w_0$$

The input features (components of the feature vector x) are:

- x_{qa} —number of questions answered in class
- x_{at} —attendance rate
- x_{hw} —homework score
- x_{te} —midterm exam score

$$\mathbf{W} = [[w_1, w_2, w_3, w_4]] \quad \mathbf{X} = \boxed{[x_{qa}, x_{at}, x_{hw}, x_{te}]}$$

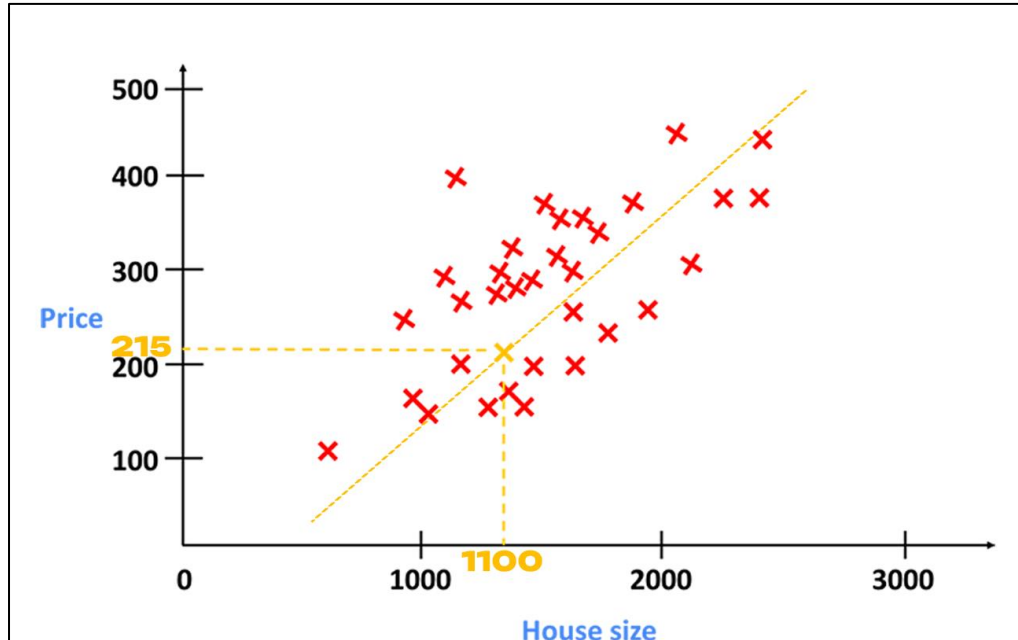

for one sample

Linear Equations

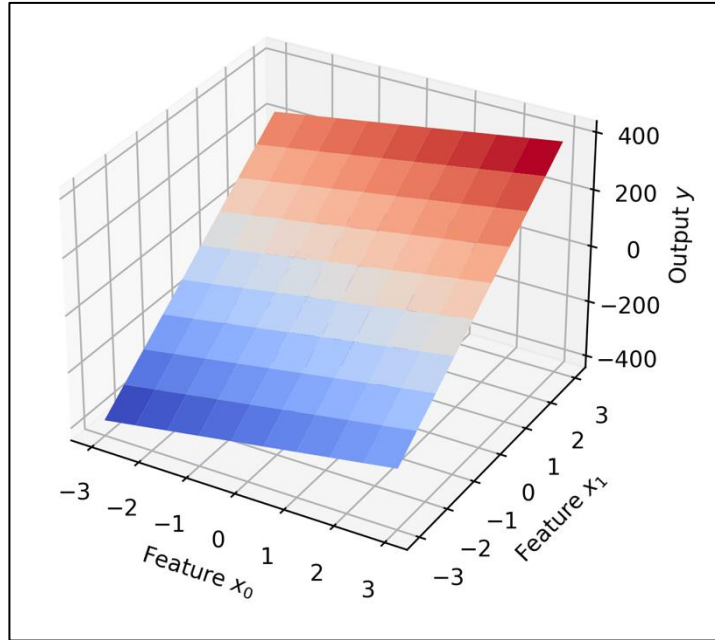
$$y = w_1 x_1 + w_0$$

For example, $y = w_1 x_{qa} + w_0$

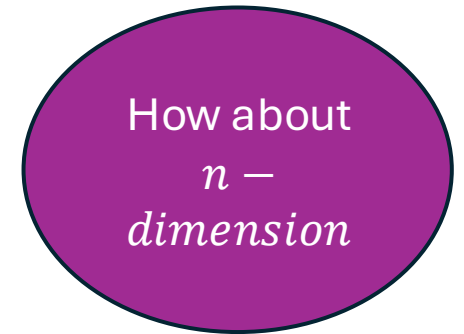
1D vs. 2D vs. n – Dimension



A Line



A plane



A hyper-plane

Linear Equations

$$y = w_1 x_{qa} + w_2 x_{at} + w_3 x_{hw} + w_4 x_{te} + w_0$$



Matrix form

$$y = \mathbf{W} \mathbf{X}^T + w_0$$

(,) (1,m) (m,1) (,)
 (1,1)=(,)



Many n samples, one output (scalar)

$$y = \mathbf{W} \mathbf{X}^T + w_0$$

(1,n) (1,m) (m,n) (,)
 (1,n)

Broadcasting w_0 to be (1,n) by cloning

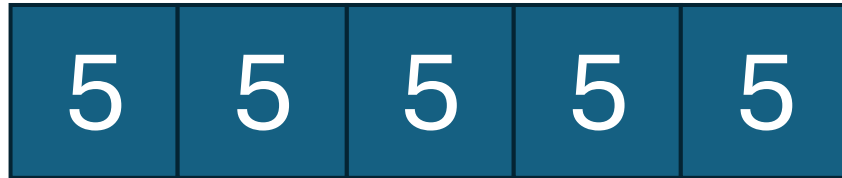
Broadcasting

5

(,)



*Broadcasting
a scalar to a
column vector
(1,5)*



(1,5)

Linear Equations

$$y = WX^T + w_0$$

$(,)$ $\underbrace{(1,m) \quad (m,1)}_{(1,1)=(,)} \quad (,)$

$(1,1)=(,)$



Many n samples

$$y = WX^T + w_0$$

$(1,n)$ $\underbrace{(1,m) \quad (m,n)}_{(1,n)} \quad (,)$

Broadcasting w_0 to be $(1,n)$ by cloning



*Many n samples,
many k outputs*

$$Y = WX^T + w_0$$

(k,n) $\underbrace{(k,m) \quad (m,n)}_{(k,n)} \quad (k,)$

Example

X (4,3)

Sample	Feature 1	Feature 2	Feature 3
$x^{(1)}$	2	4	6
$x^{(2)}$	5	1	3
$x^{(3)}$	1	2	8
$x^{(4)}$	3	7	4

W (2,3)

$$\begin{bmatrix} 0.5 & 0.2 & 0.1 \\ 1.0 & 0.3 & 0.4 \end{bmatrix}$$

W₀ (2,)

$$\begin{bmatrix} 10 \\ 20 \end{bmatrix}$$

Example

X (4,4)

Sample	Bias Feature	Feature 1	Feature 2	Feature 3
$x^{(1)}$	1	2	4	6
$x^{(2)}$	1	5	1	3
$x^{(3)}$	1	1	2	8
$x^{(4)}$	1	3	7	4

W (2,4)

Augmented columns

Depending our libraries, this part is usually done for you behind the scene.

$$\tilde{W} = [w_0 \quad W] = \begin{bmatrix} 10 & 0.5 & 0.2 & 0.1 \\ 20 & 1.0 & 0.3 & 0.4 \end{bmatrix}$$

Linear Equations

$$y = WX^T$$

(,) (1,m) (m,1)

(1,1)=(,)



Many n samples

$$y = WX^T$$

(1,n) (1,m) (m,n)

(1,n)

Broadcasting w_0 to be (1,n) by cloning



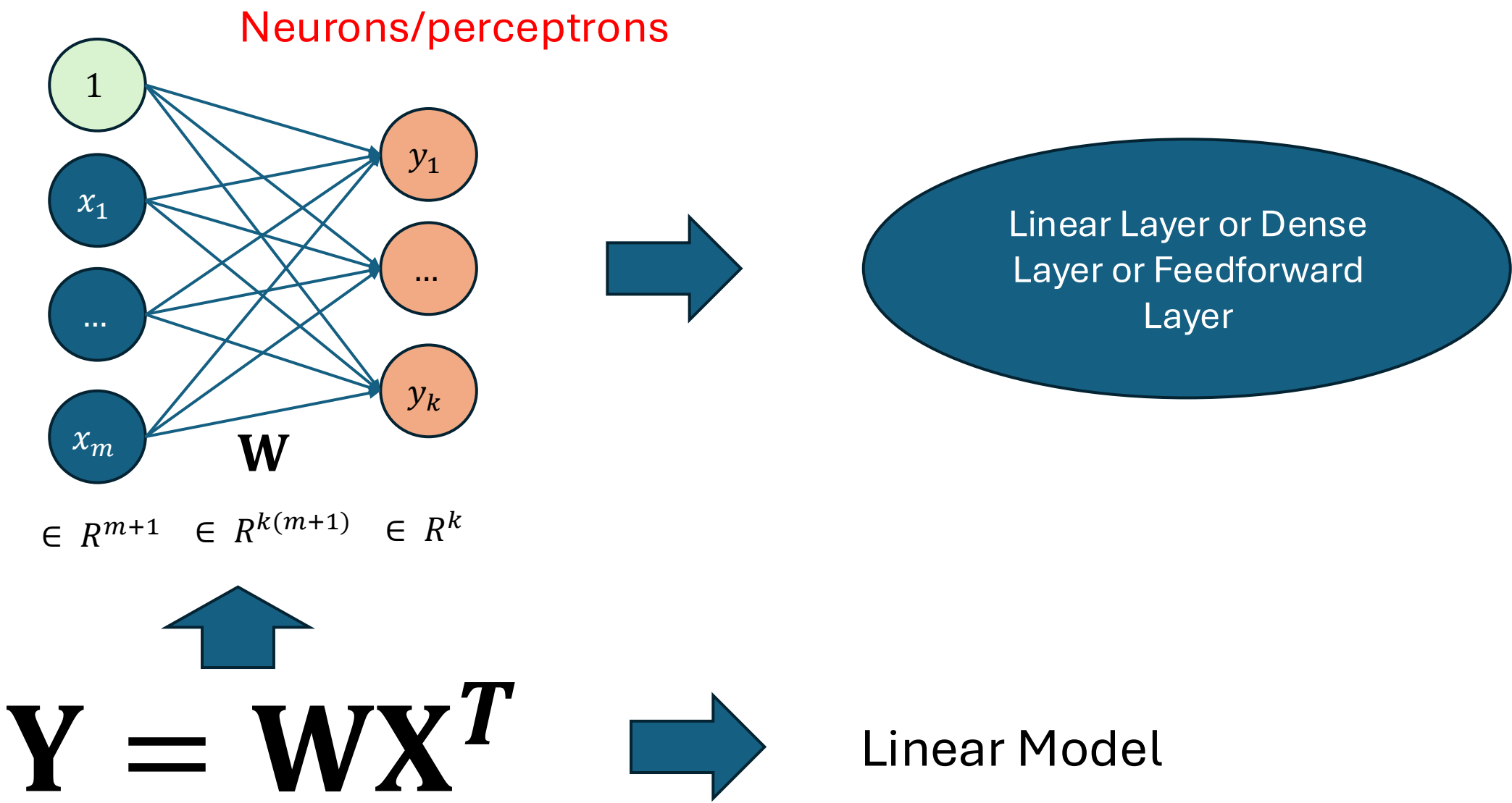
*Many n samples,
many k outputs*

$$Y = WX^T$$

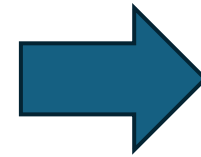
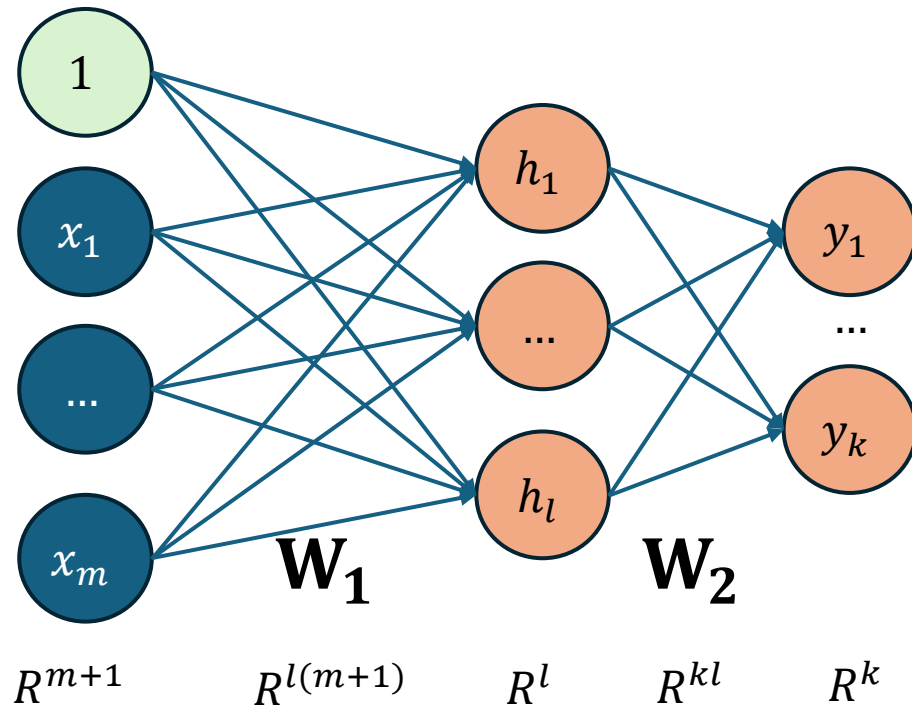
(k,n) (k,m) (m,n)

(k,n)

Computational Graph for One Sample

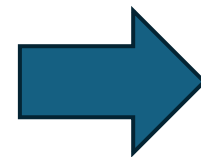


Computational Graph for One Sample



Simple Neural Network
or Multi-layered
Perceptron Network

$$Y = W_2(W_1 X^T)$$



Linear Model

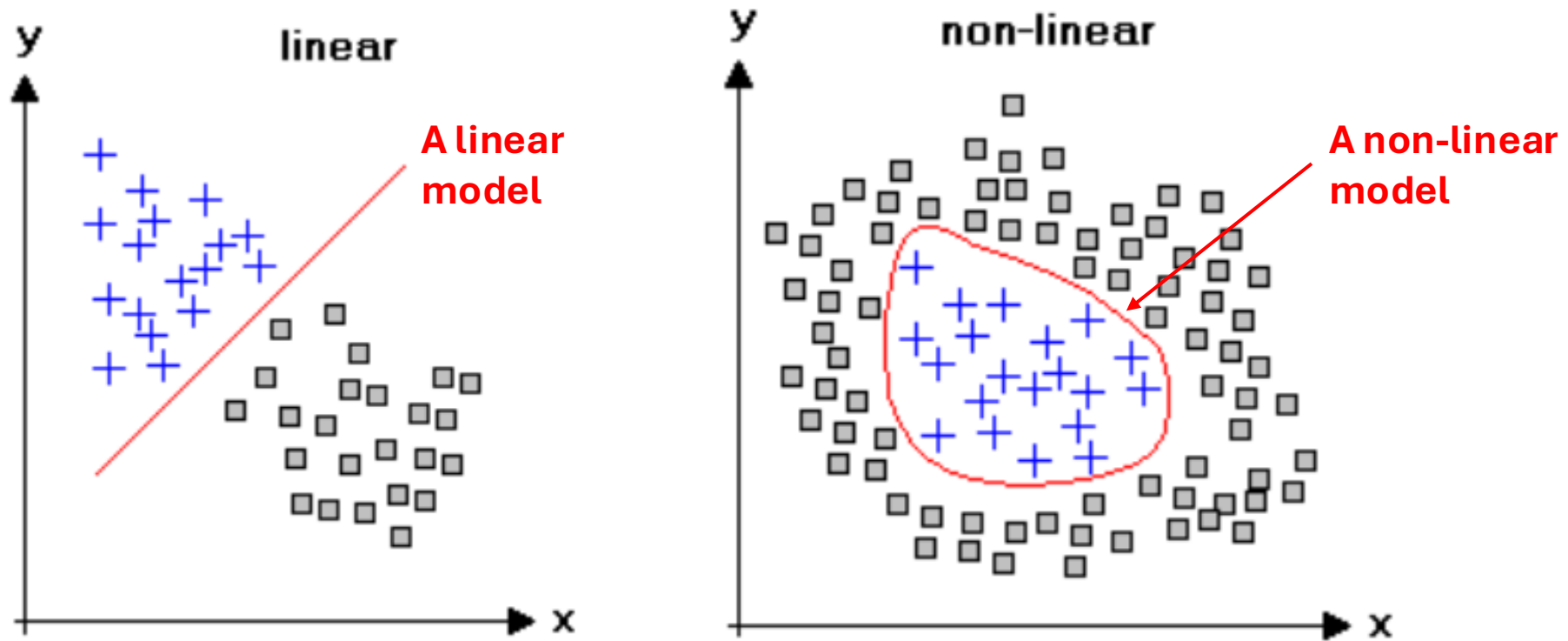
Linear Definition

A linear function :

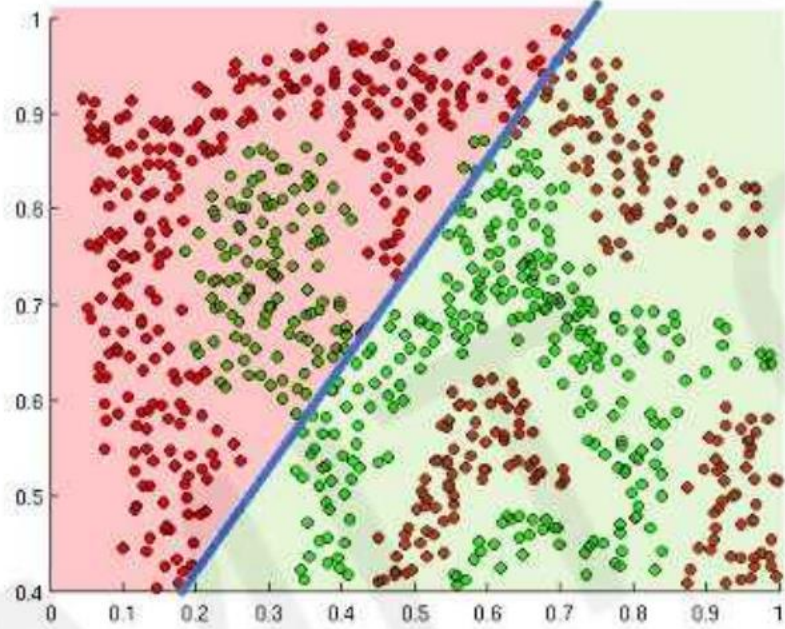
1. addition: $f(x + y) = f(x) + f(y)$

2. scaling: $f(ax) = af(x)$

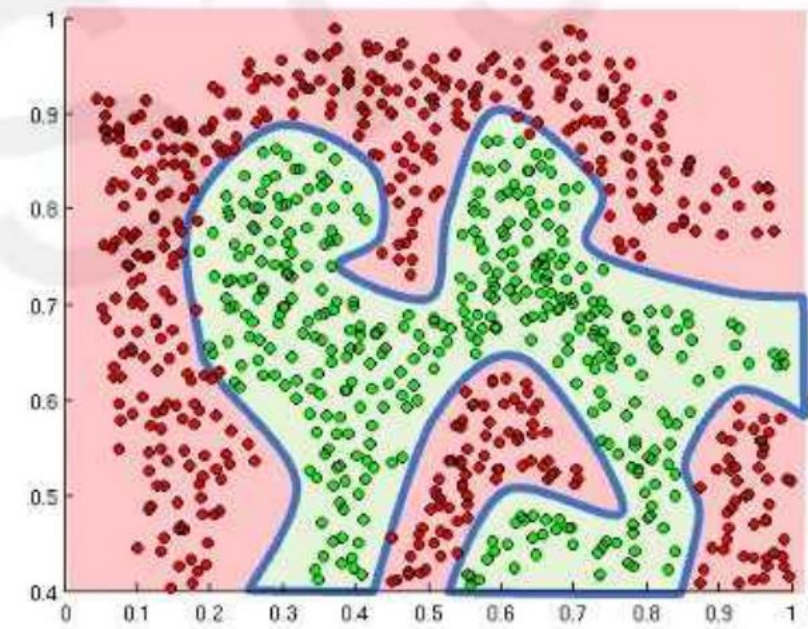
Linearity vs. Non-Linearity



How to Introduce Non-Linearities

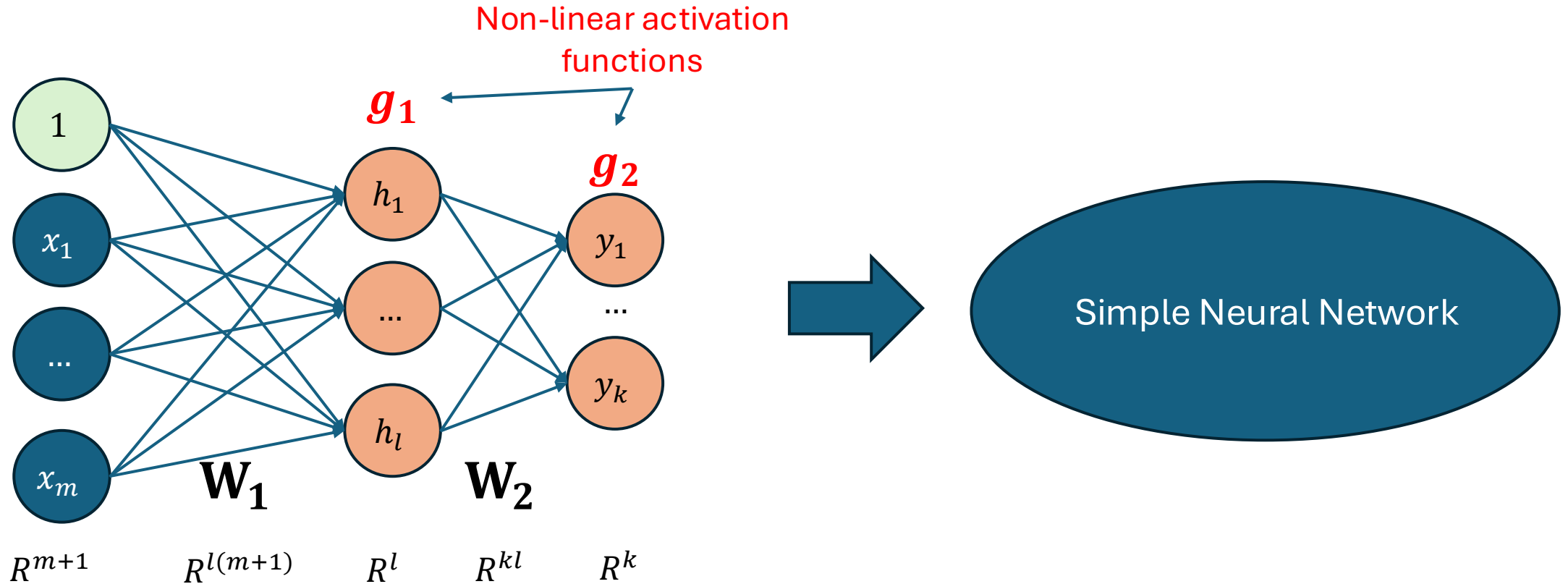


Linear activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

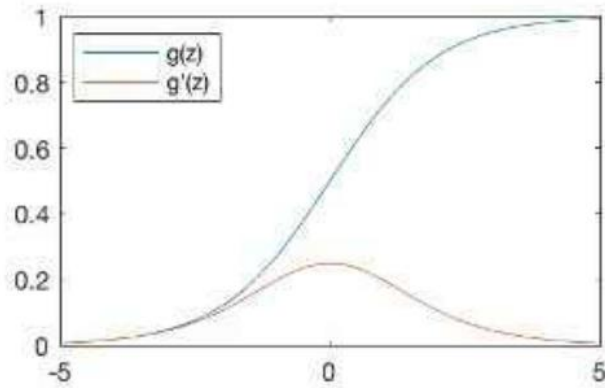
Computational Graph for One Sample



$$\mathbf{Y} = \mathbf{g}_2(\mathbf{W}_2 \mathbf{g}_1(\mathbf{W}_1 \mathbf{X}^T)) \Rightarrow \text{Non-Linear Model}$$

Common Activation Functions

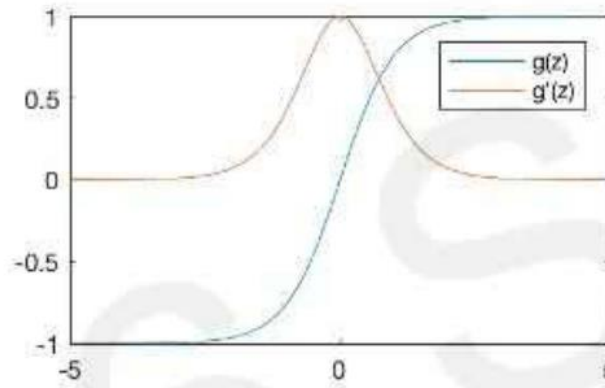
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

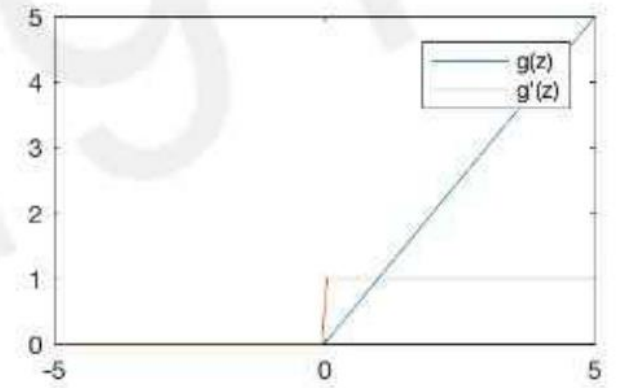
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)

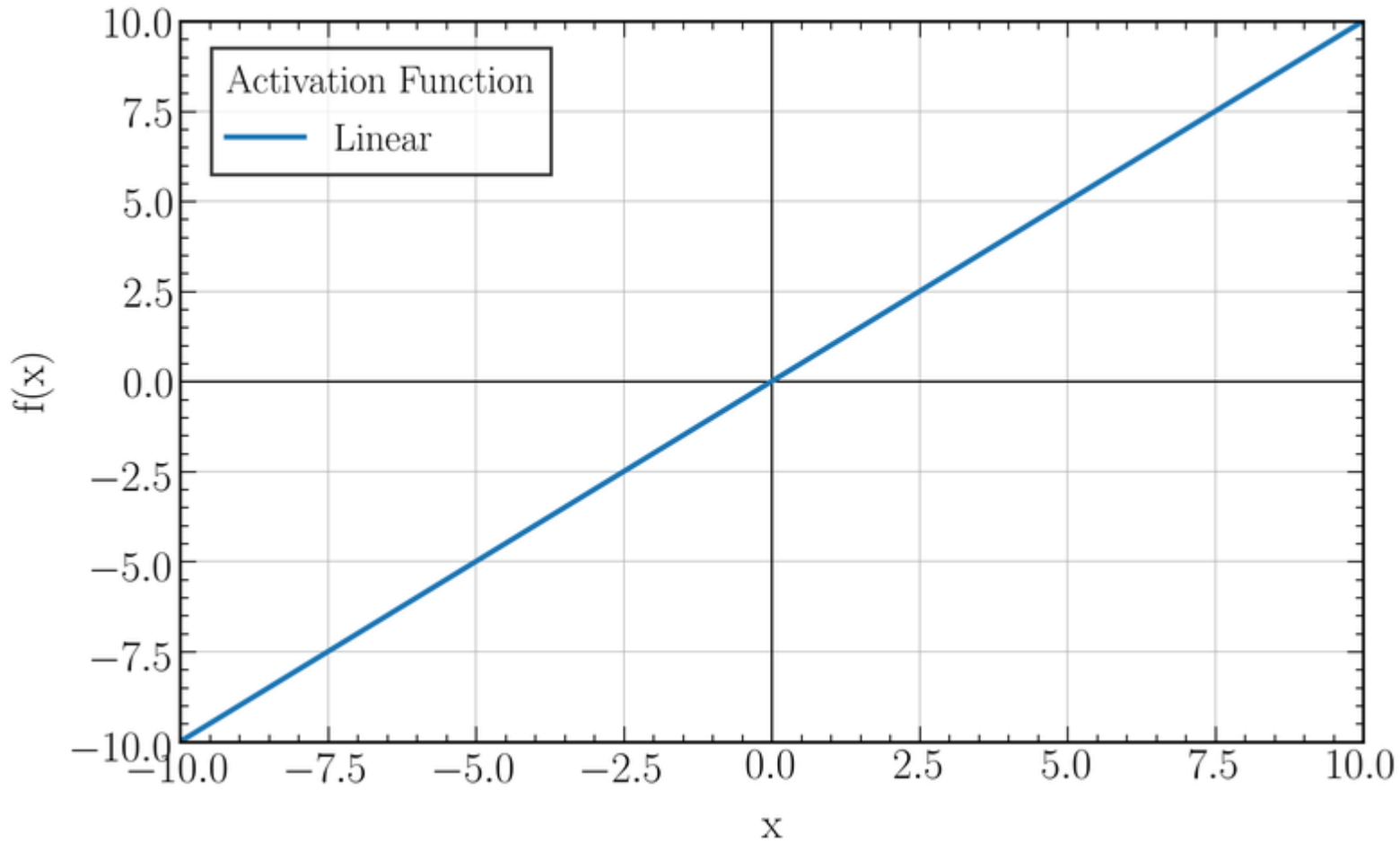


$$g(z) = \max(0, z)$$

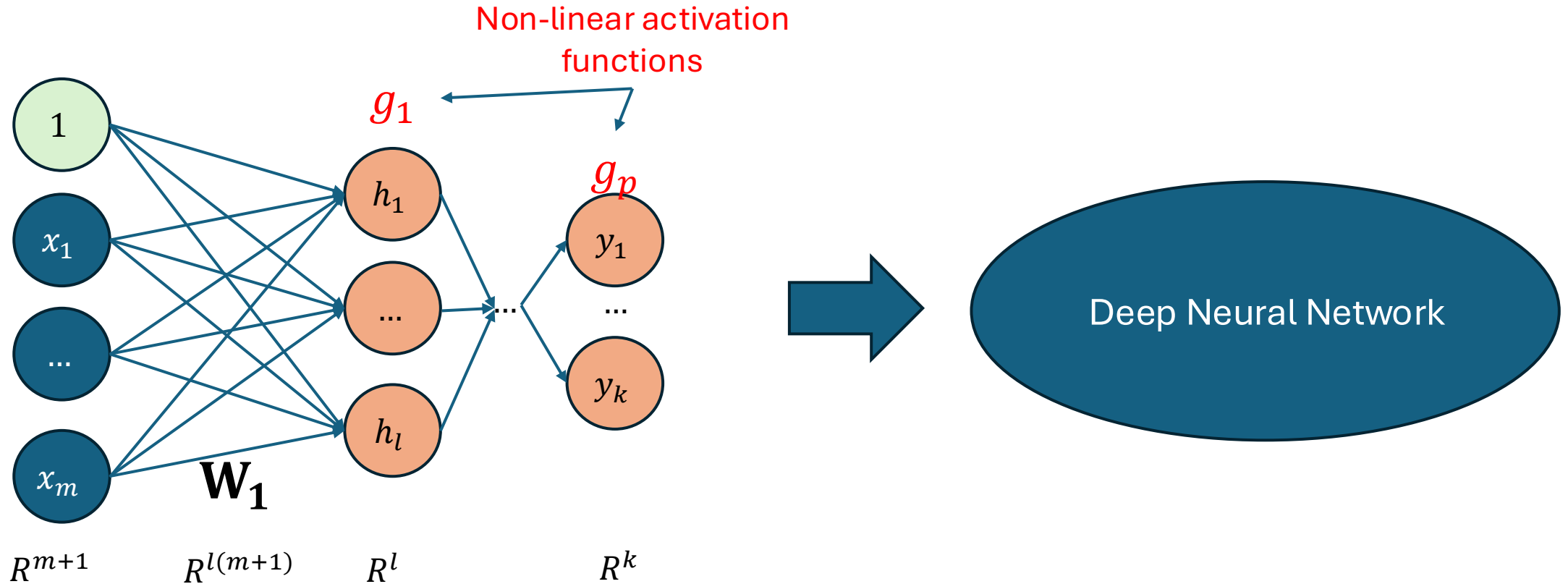
$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

Common Activation Functions

$$f(x) = x$$

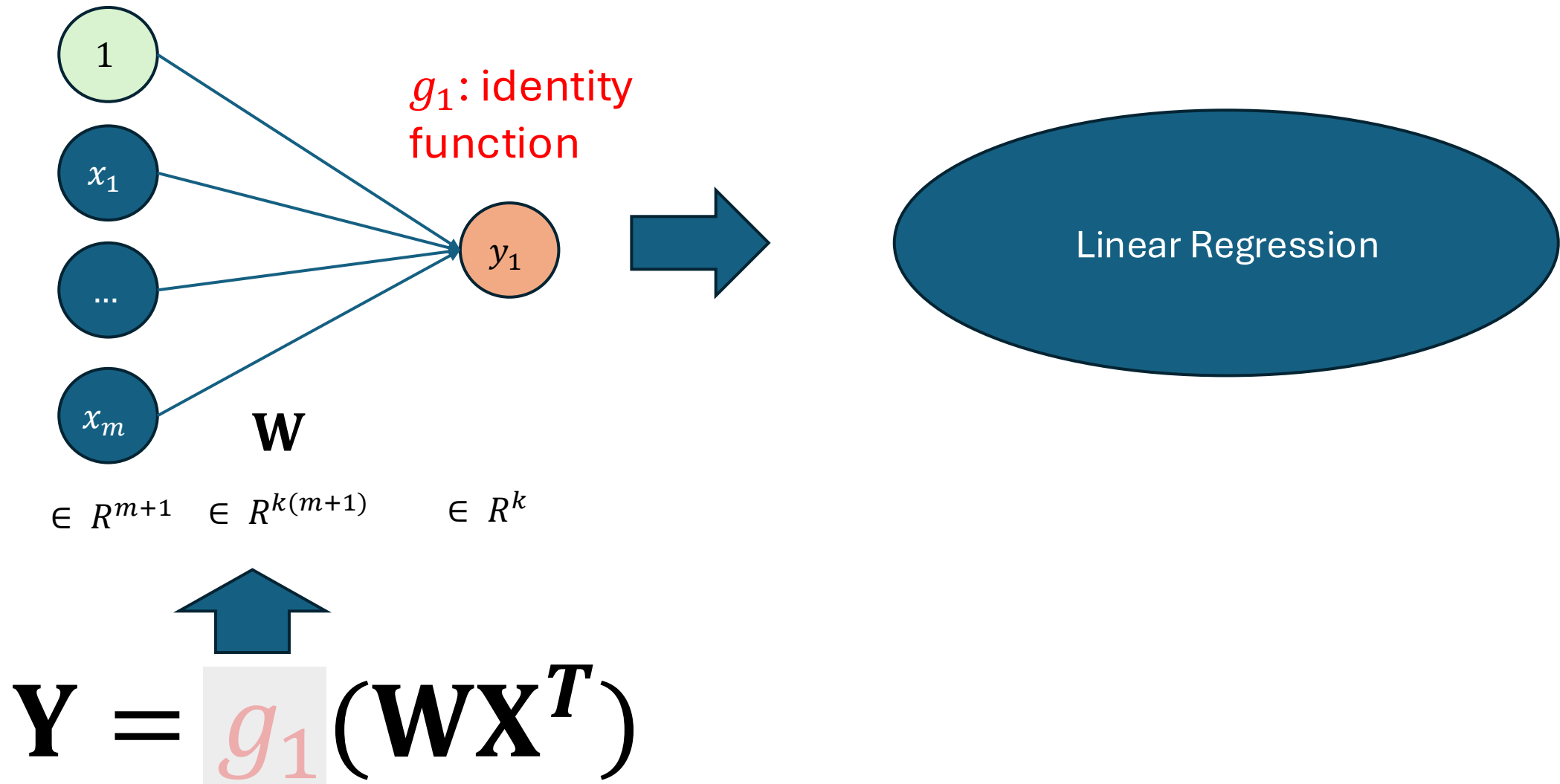


Computational Graph for One Sample

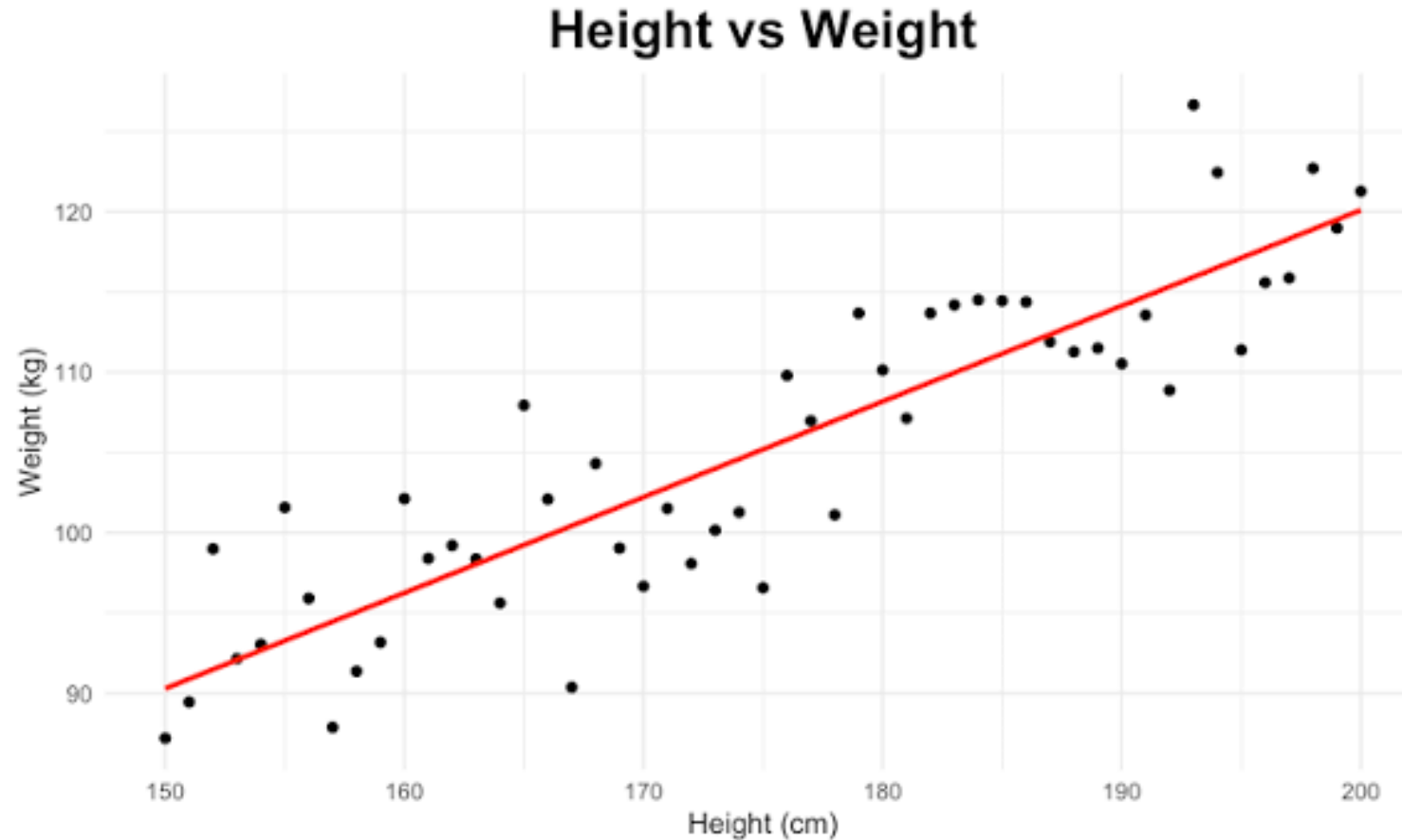


$$\mathbf{Y} = g_p(\dots g_1(\mathbf{W}_1 \mathbf{X}^T)) \Rightarrow \text{Non-Linear Model}$$

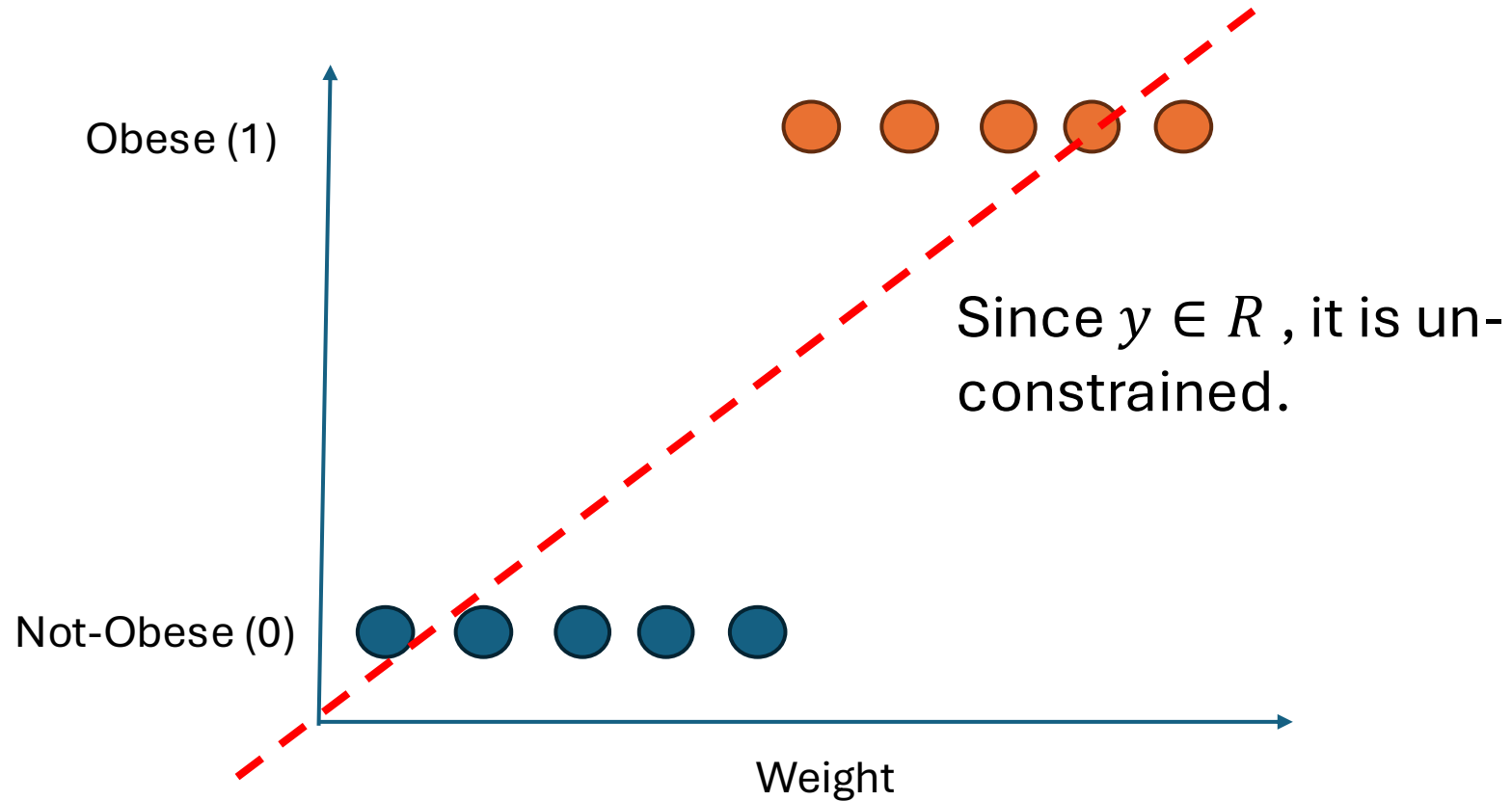
Linear Regression as a Special Linear Layer



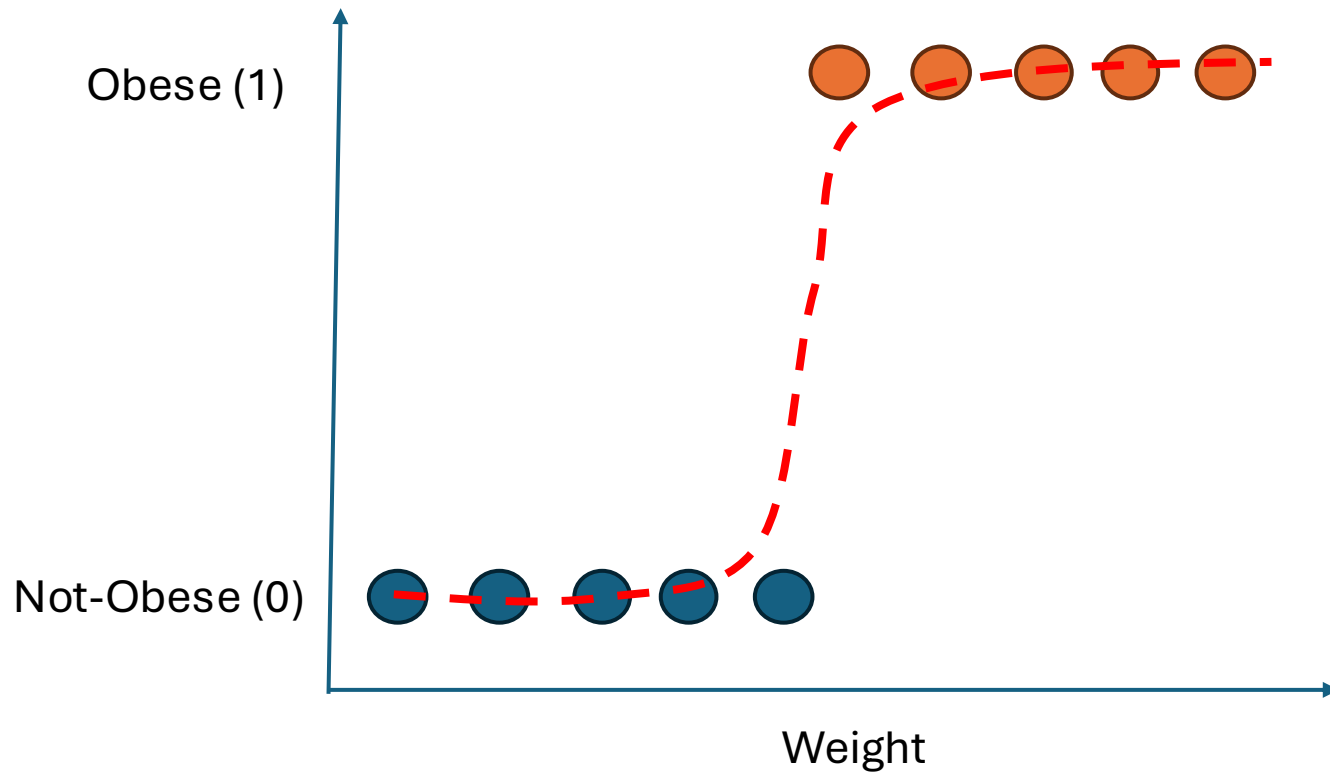
Linear Regression as a Special Linear Layer



Logistic Regression as a Special Linear Layer



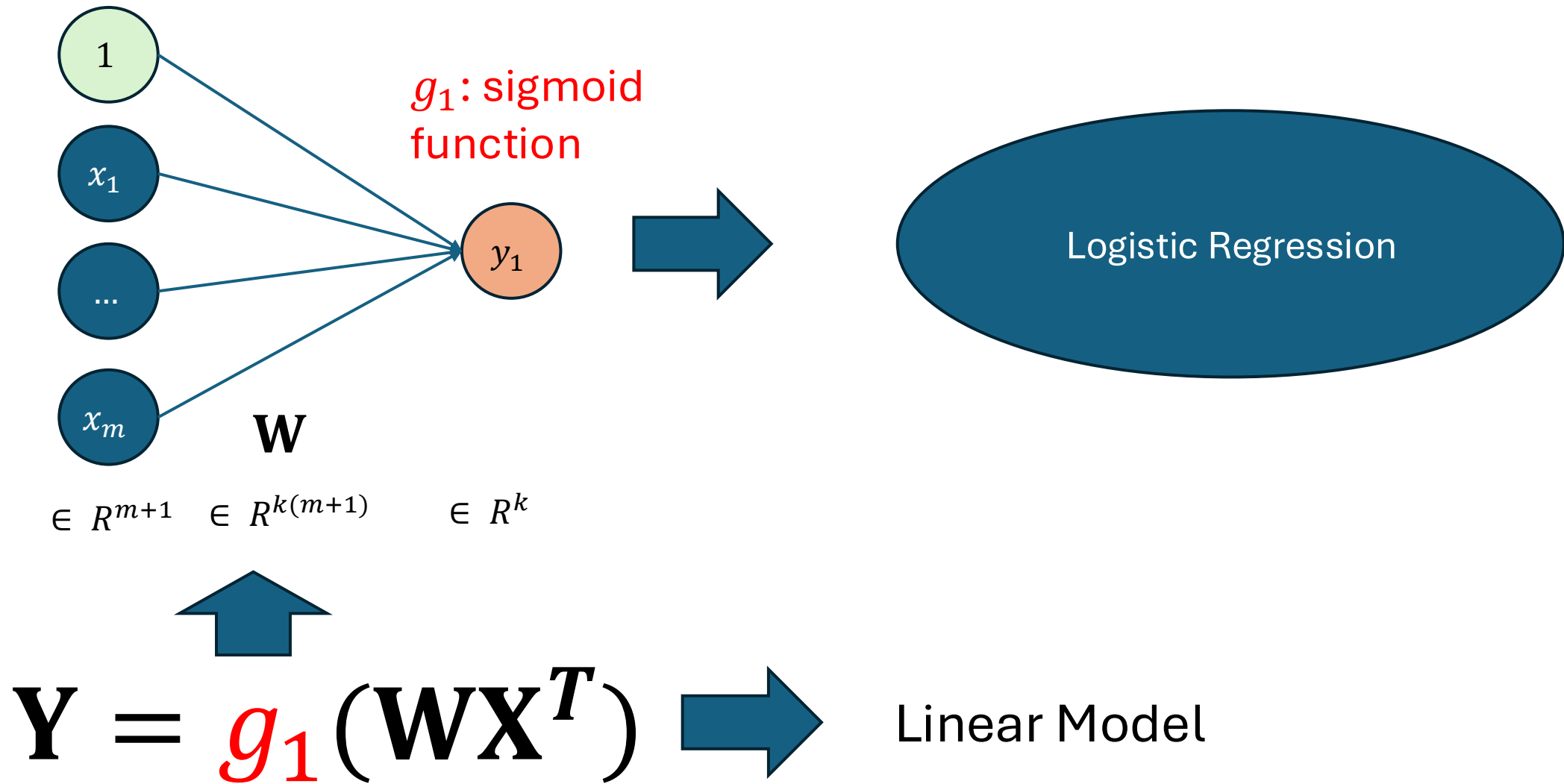
Logistic Regression as a Special Linear Layer



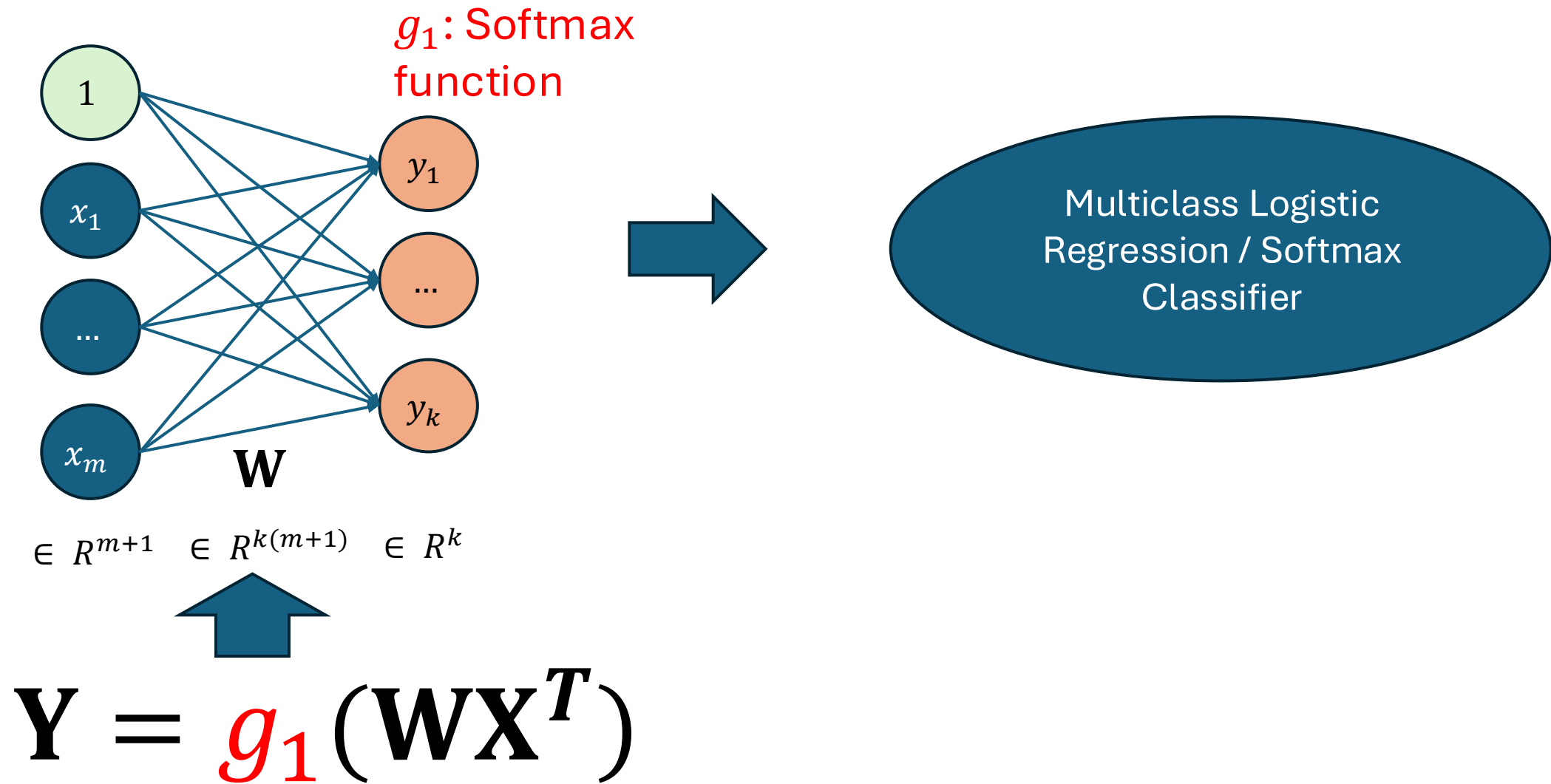
What if we predict
 $p(\text{obese or } 1)$?

$$y = \begin{cases} 1 \text{ or obese if } p \geq 0.5 \\ 0 \text{ or not obese otherwise} \end{cases}$$

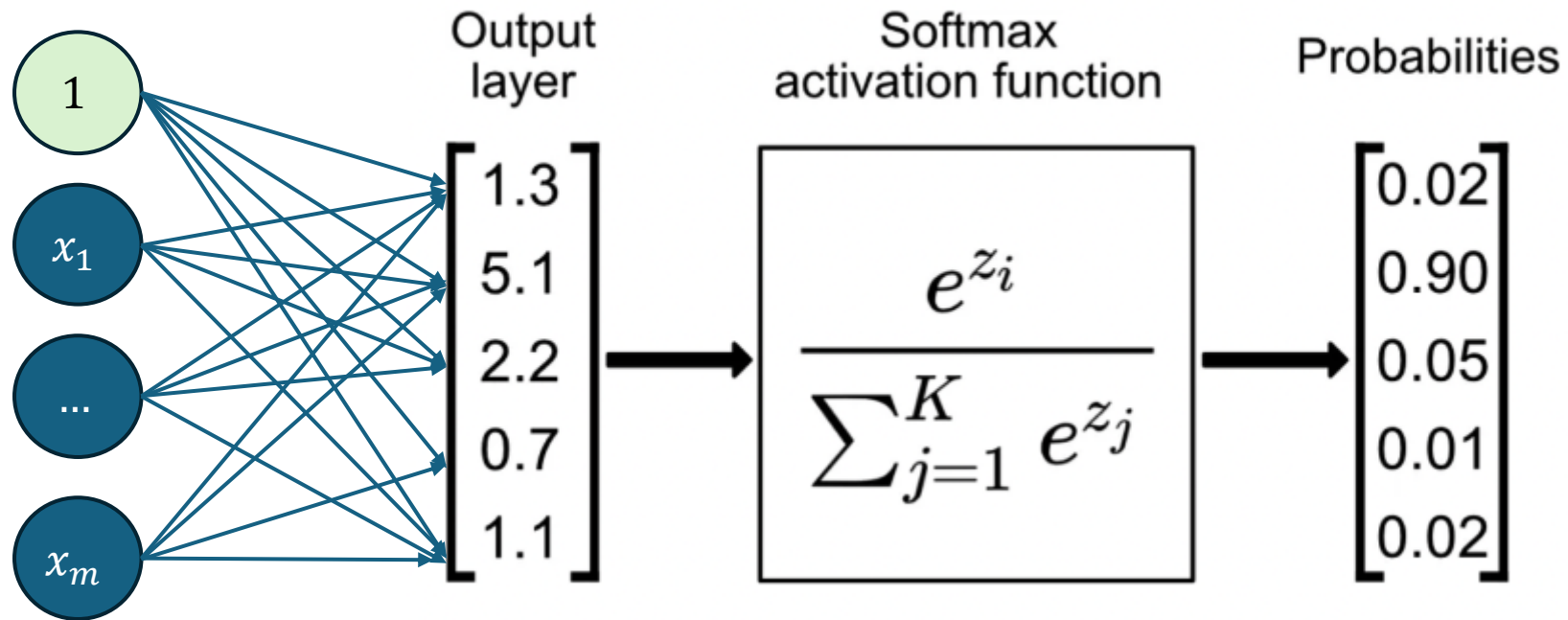
Logistic Regression as a Special Linear Layer



Multiclass Logistic Regression / Softmax Classifier as a Special Linear Layer



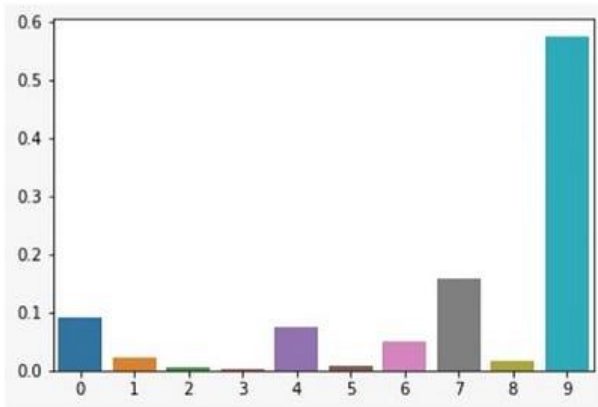
Softmax Activation Function



Softmax Activation Function

SOFTMAX WITHOUT TEMPERATURE ($T=1$)

$$\frac{e^{z_i}}{\sum_j e^{z_j}}$$



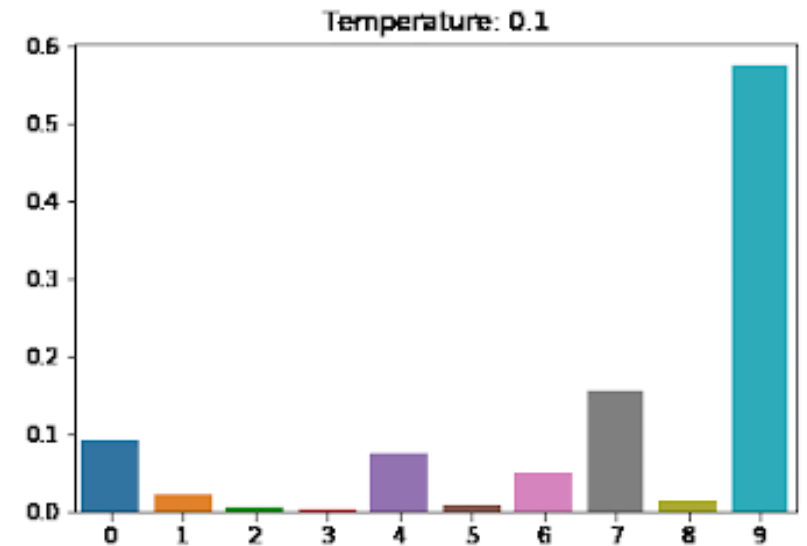
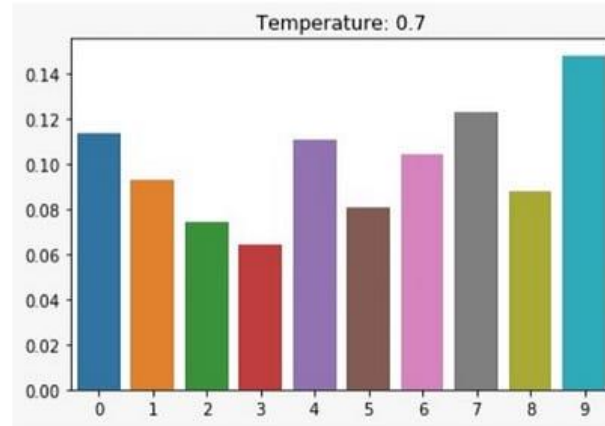
LESS ENTROPY

INCREASE IN ENTROPY
WITH INCREASE IN T

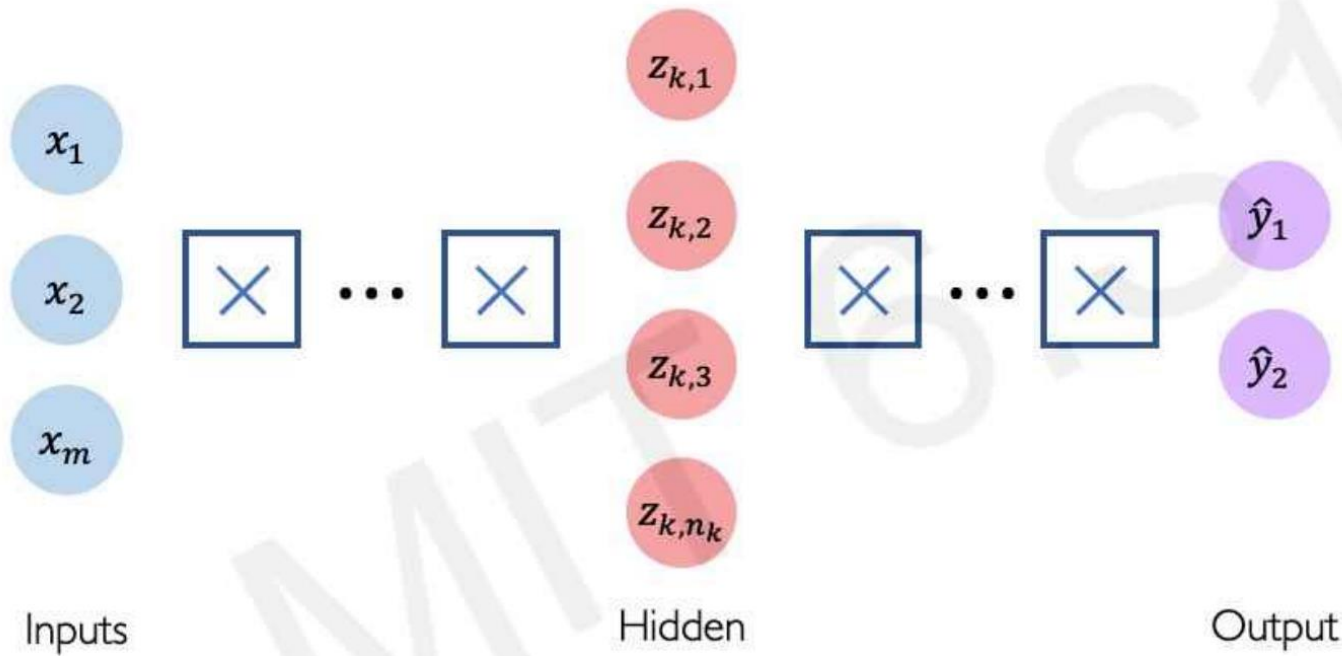
MORE ENTROPY

SOFTMAX WITH TEMPERATURE

$$\frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

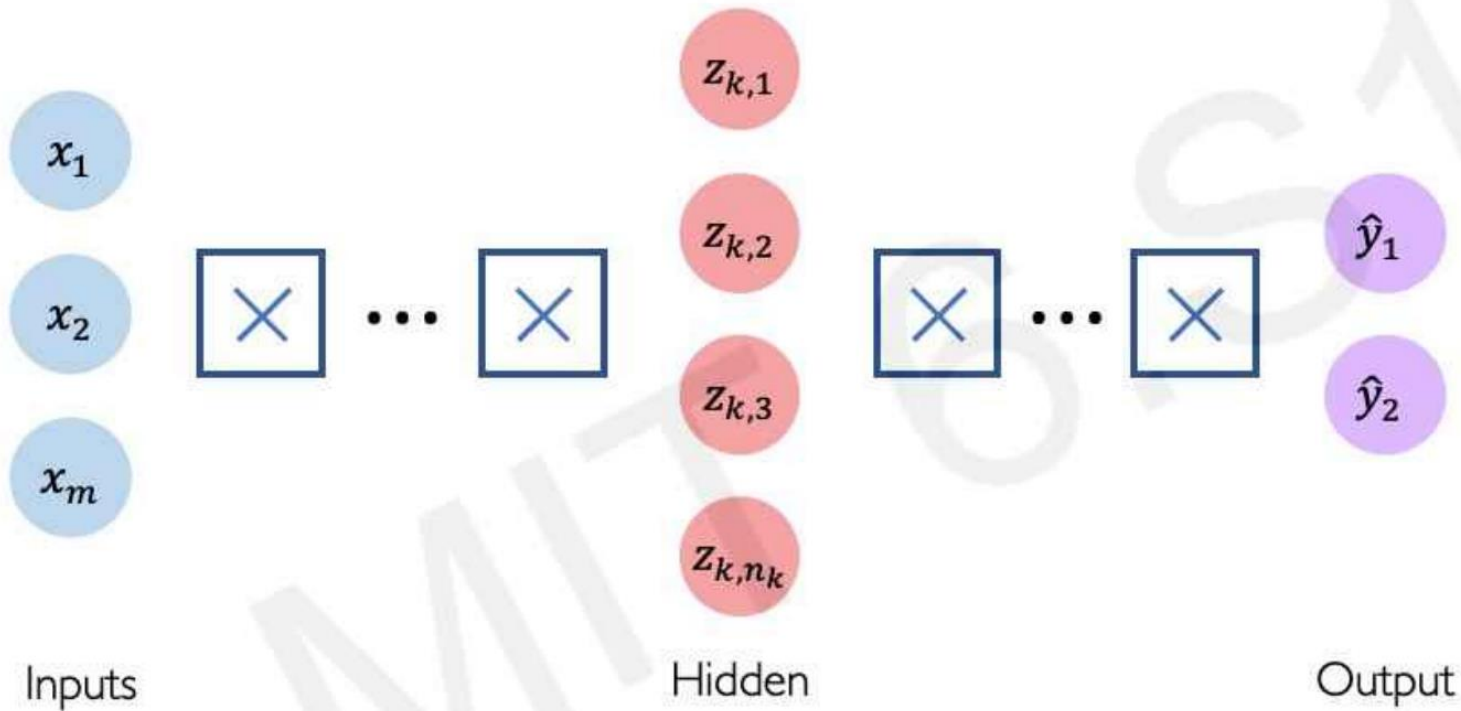


Deep Neural Networks – Design



- How many input features - m
- How many hidden layers - n_{layers}
- How many hidden neurons in each layer ? – $[H_1, \dots, H_l]$
- Each layer's activation function?
- Output activation – Sigmoid (binary classification), Softmax (multi-class classification), Identity (regression)

Deep Neural Networks – Design in Practice



```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(n1),
    tf.keras.layers.Dense(n2),
    :
    tf.keras.layers.Dense(2)
])
```



```
from torch import nn

model = nn.Sequential(
    nn.Linear(m, n1),
    nn.ReLU(),
    :
    nn.ReLU(),
    nn.Linear(nK, 2)
)
```

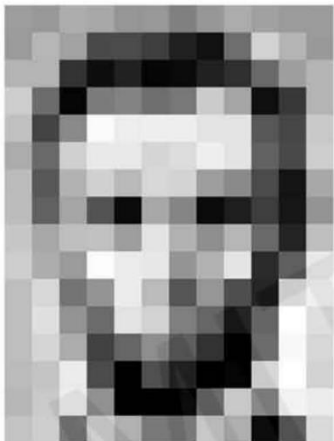
Pre-Processing

Structured Data

Instances		Features					Targets	
		age	workclass	education	marital-status	sex	hours-per-week	income
→	39	State-gov	Bachelors	Never-married	Male	40	<=50K	
→	50	Self-emp-not-inc	Bachelors	Married-civ-spouse	Male	13	<=50K	
→	38	Private	HS-grad	Divorced	Male	40	<=50K	
→	53	Private	11th	Married-civ-spouse	Male	40	<=50K	
→	28	Private	Bachelors	Married-civ-spouse	Female	40	<=50K	
→	37	Private	Masters	Married-civ-spouse	Female	40	<=50K	
→	49	Private	9th	Married-spouse-absent	Female	16	<=50K	
→	52	Self-emp-not-inc	HS-grad	Married-civ-spouse	Male	45	>50K	
→	31	Private	Masters	Never-married	Female	50	>50K	
→	42	Private	Bachelors	Married-civ-spouse	Male	40	>50K	
→	37	Private	Some-college	Married-civ-spouse	Male	80	>50K	
→	30	State-gov	Bachelors	Married-civ-spouse	Male	40	>50K	

- Non-numerical inputs (i.e., categorical)
 - Nominal (no order)
 - Ordinal (order)
 - Binary
- Numeric inputs
 - Standardization
 - Min-max normalization

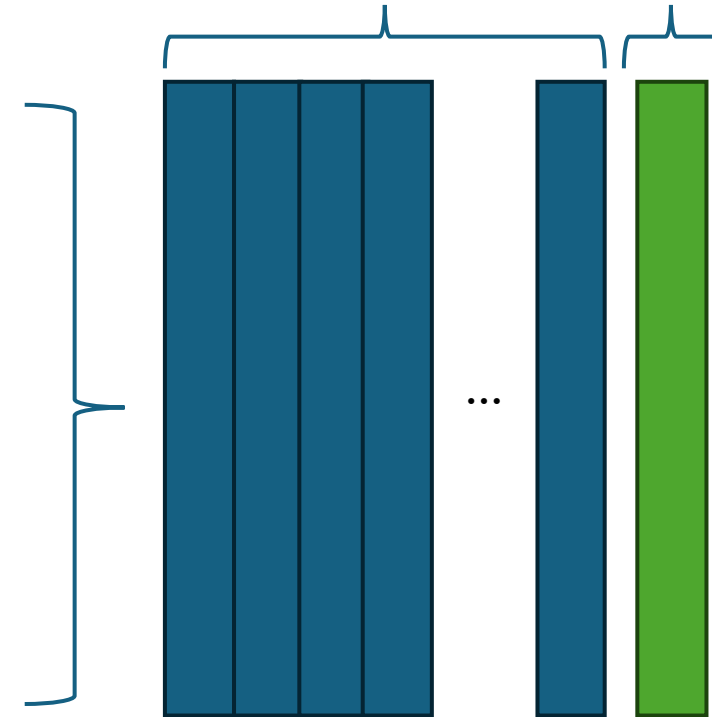
Unstructured Data



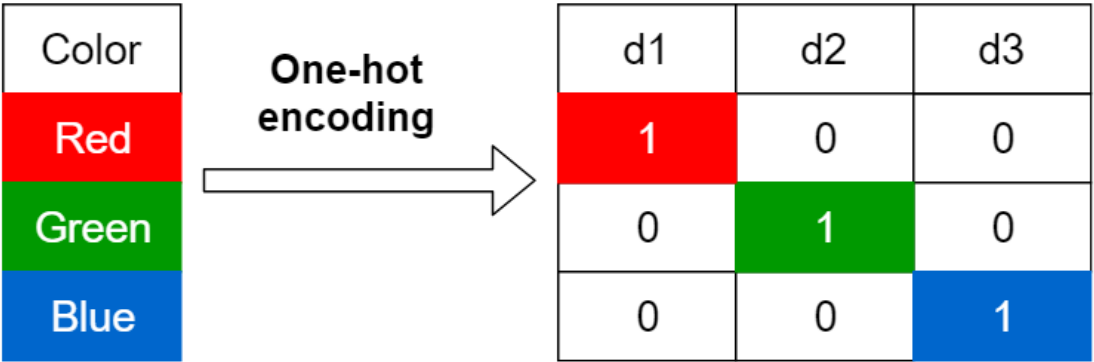
I go to school.

- Flattening to vector
- Feature extraction/processing
- Numeric inputs
 - Standardization
 - Min-max normalization

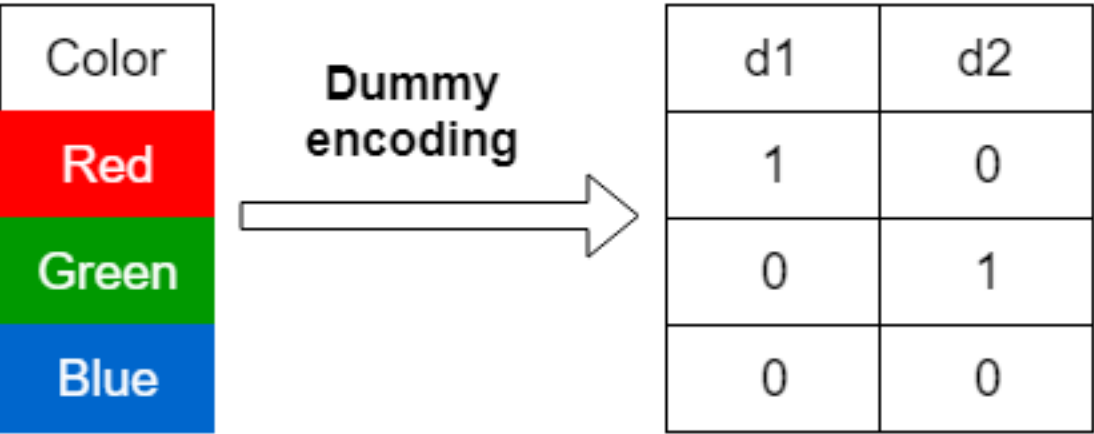
Feature Matrix, X Label, y



Non-Numerical Inputs (Categorical)



Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

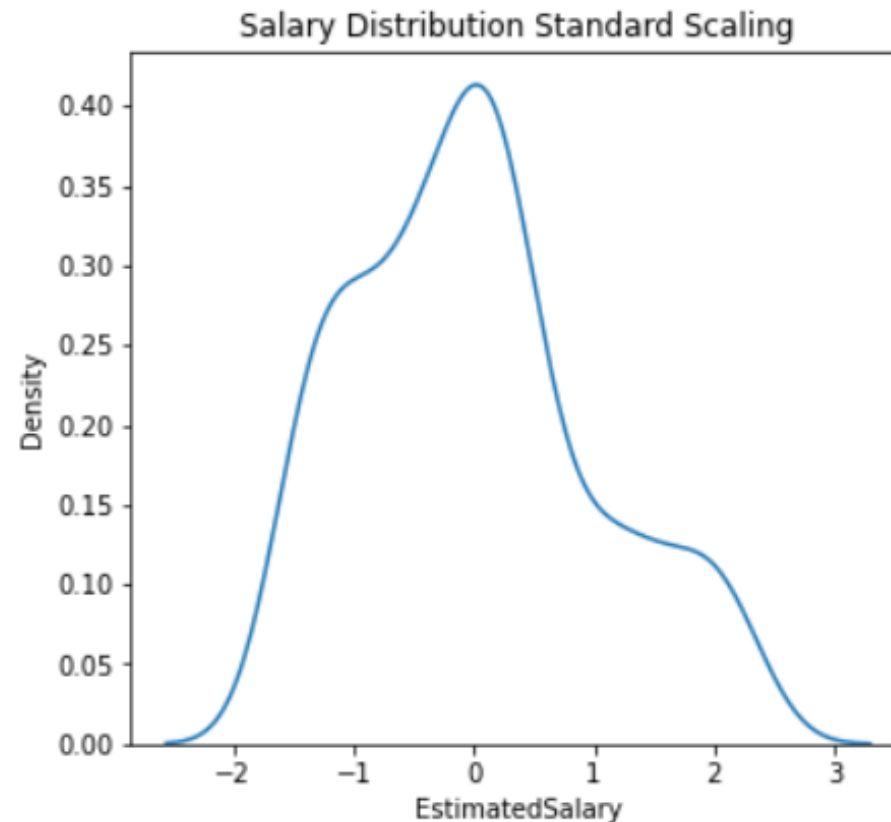
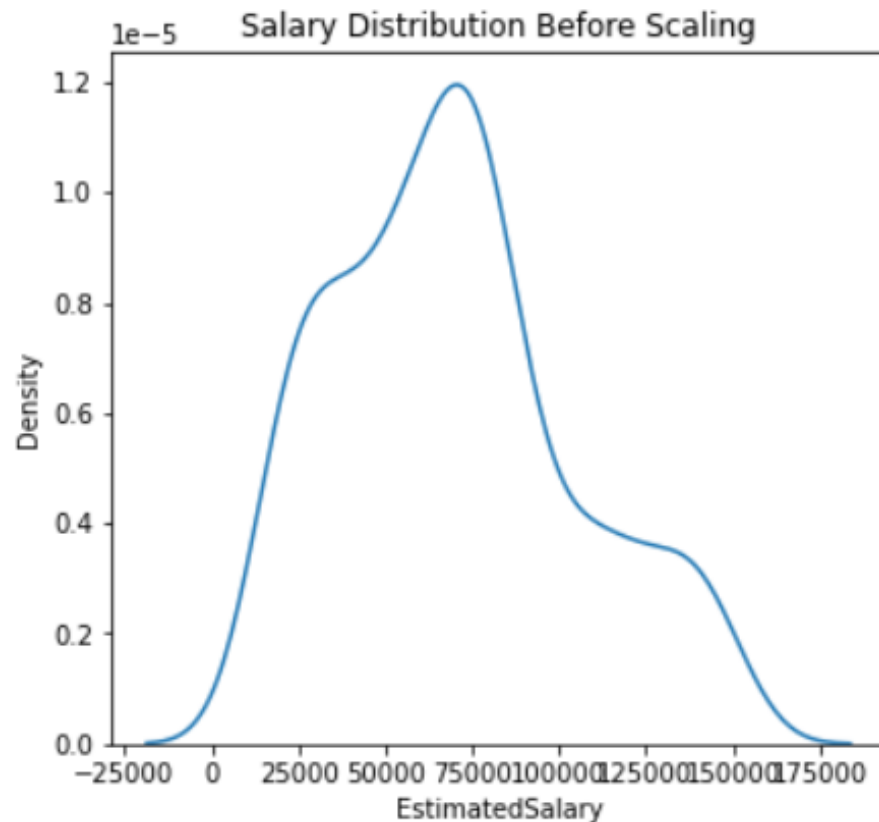


Binary Encoding	Encoded
Yes	1
No	0
...	
No	0

Numeric Inputs – Standardization

- **Concept:** Rescales the data so that it has a mean of 0 and a standard deviation of 1. It centers the data around zero.

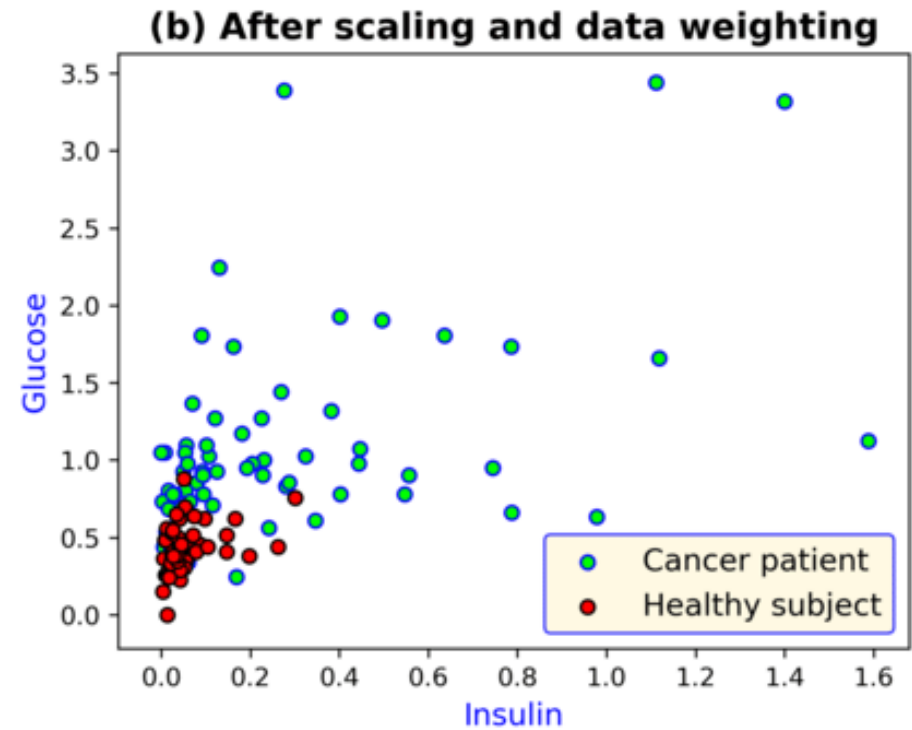
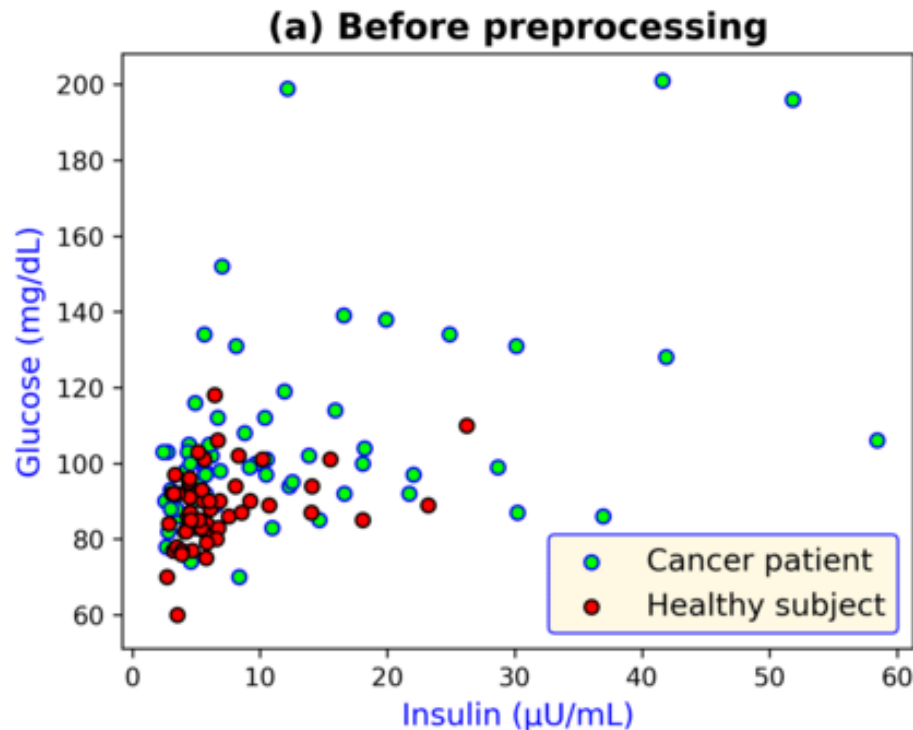
$$z = \frac{x - \mu}{\sigma}$$



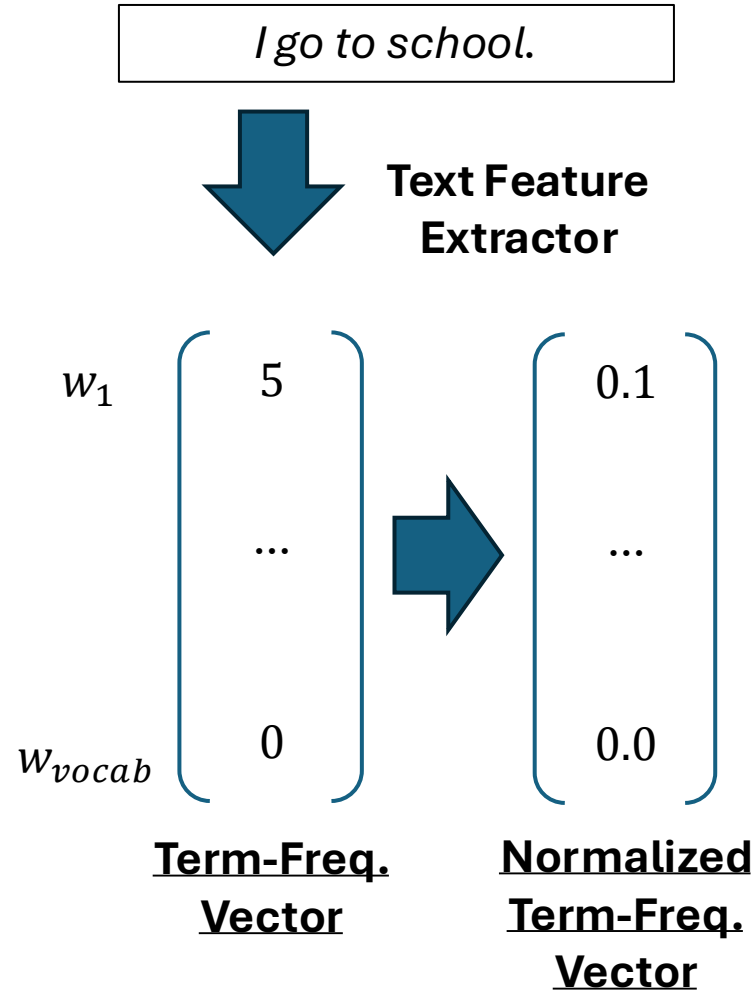
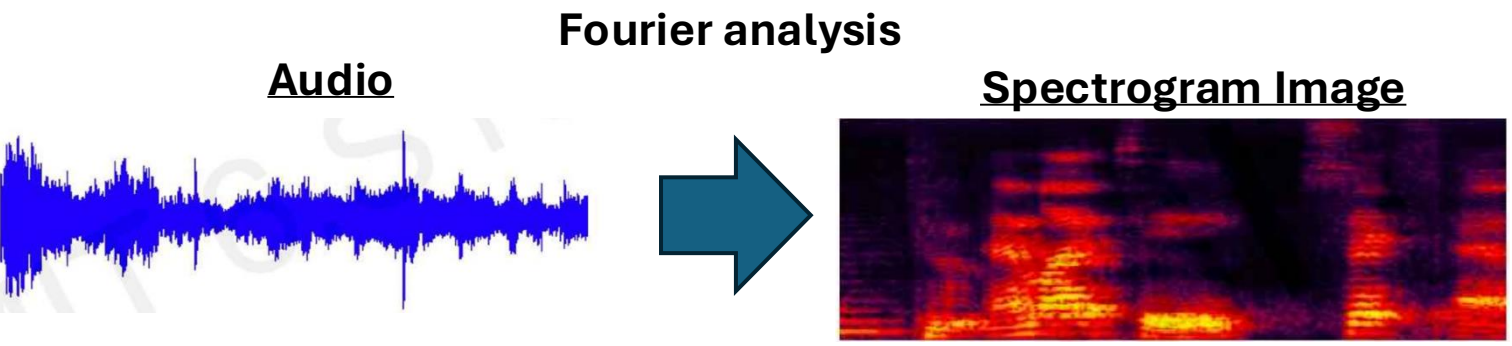
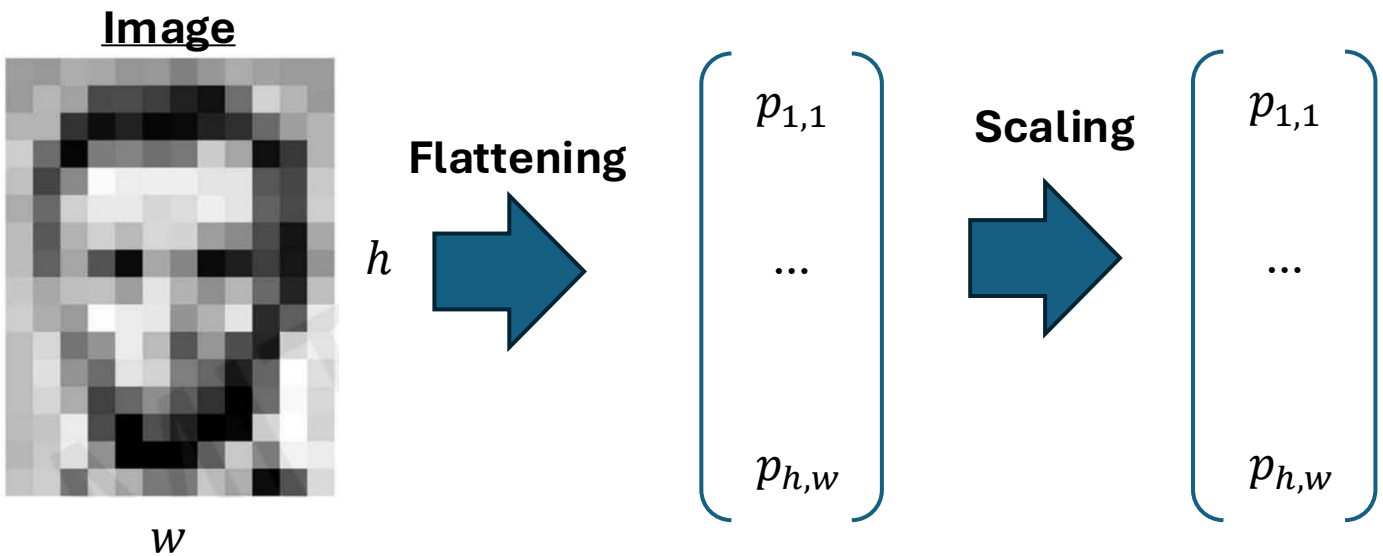
Numeric Inputs – Min-max

- **Concept:** Rescales the data so that all values fall exactly within a specific bounded range, usually between 0 and 1.

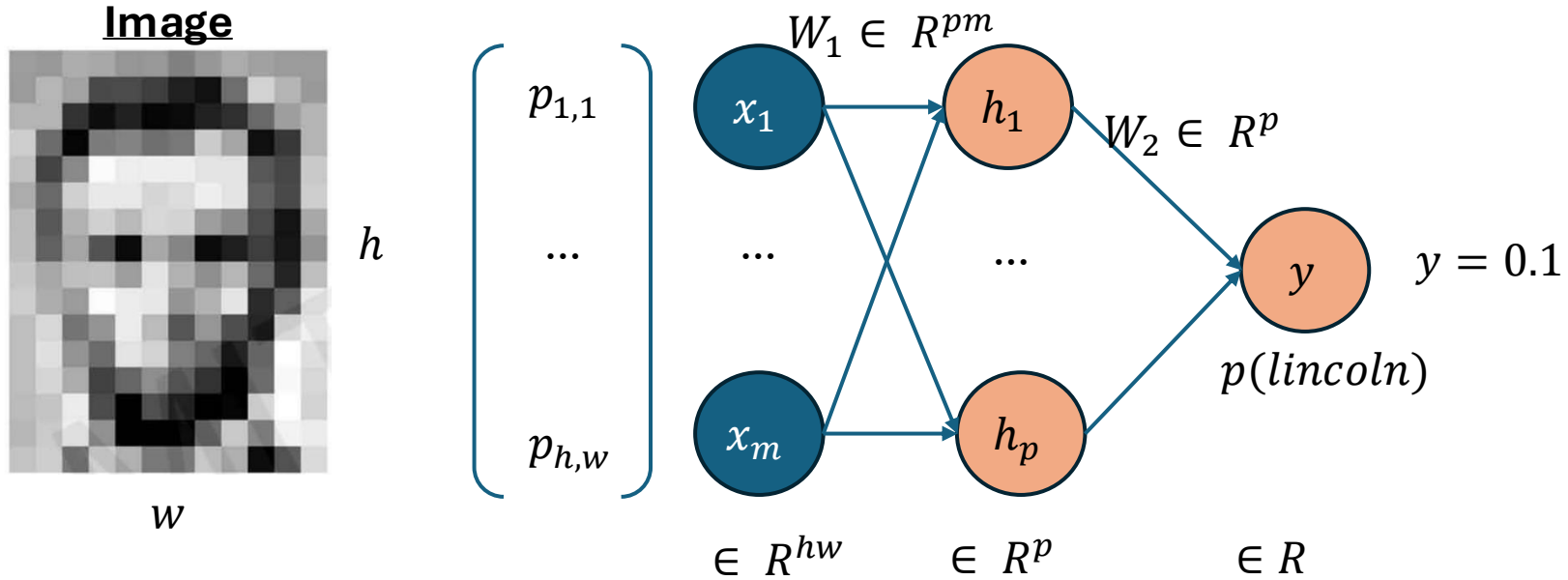
$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$



Feature Extraction



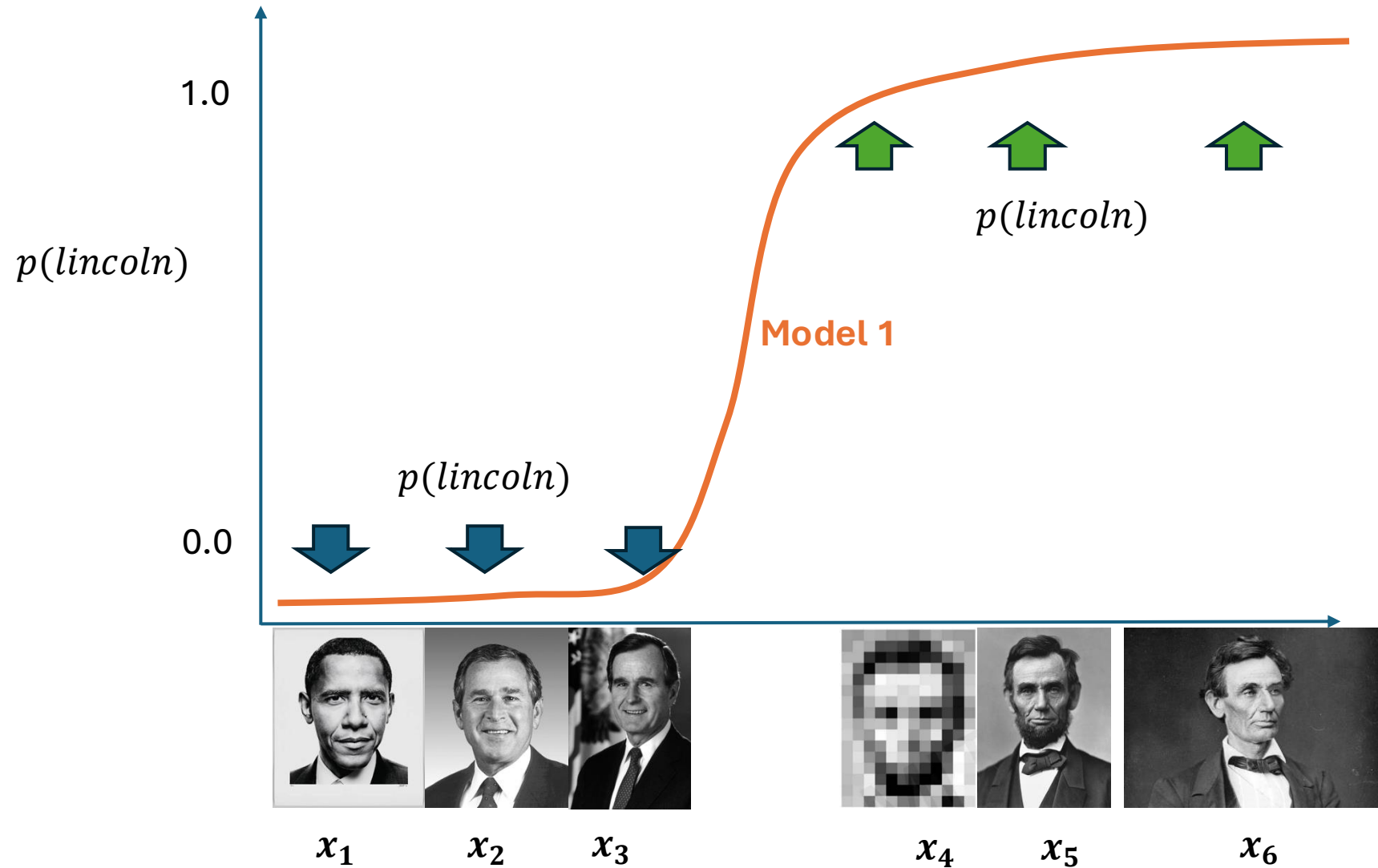
Lincoln Classifier



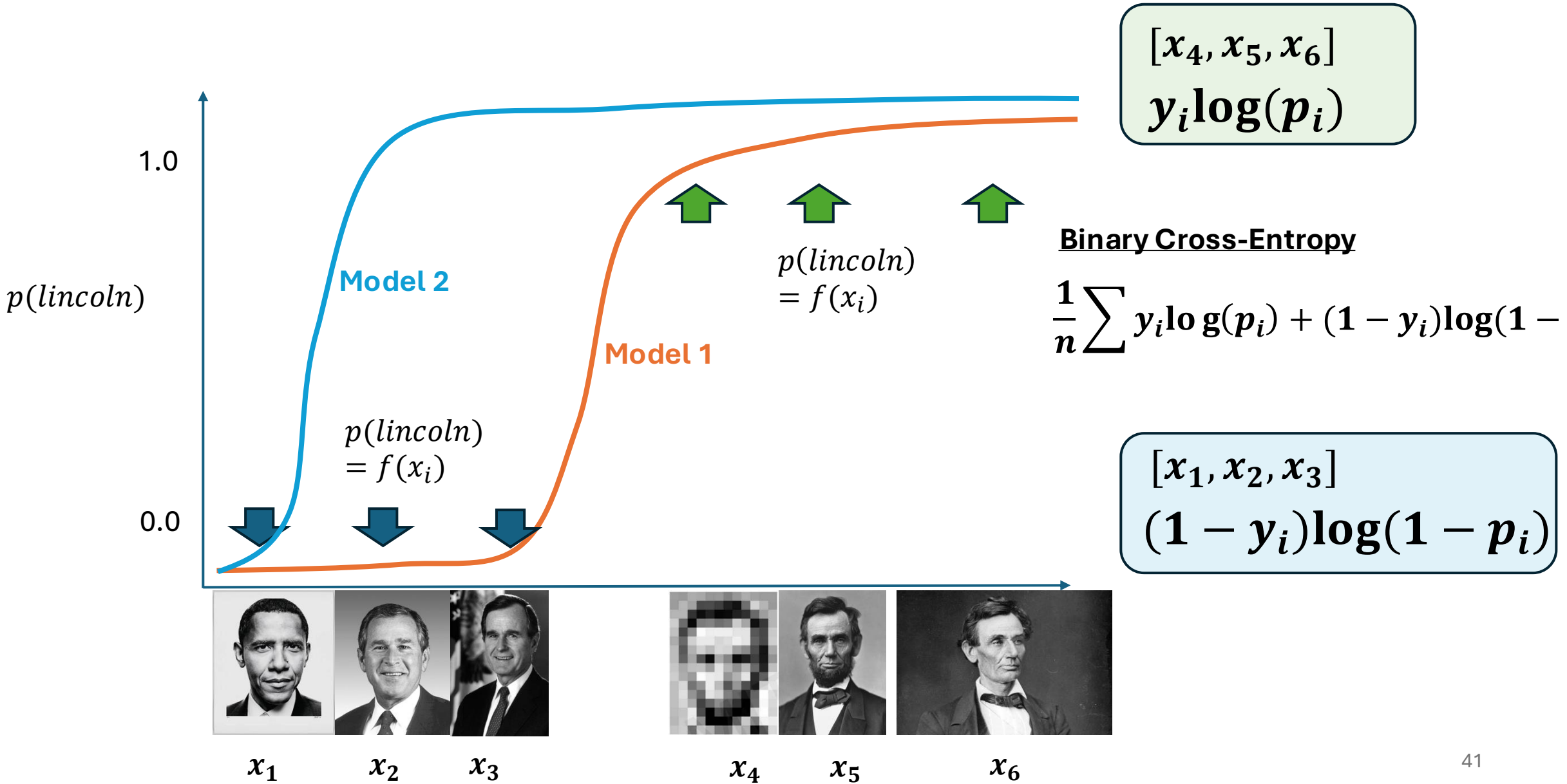
$$W_1 \in R^{pm} \sim N(0,1)$$

$$W_2 \in R^p \sim N(0,1)$$

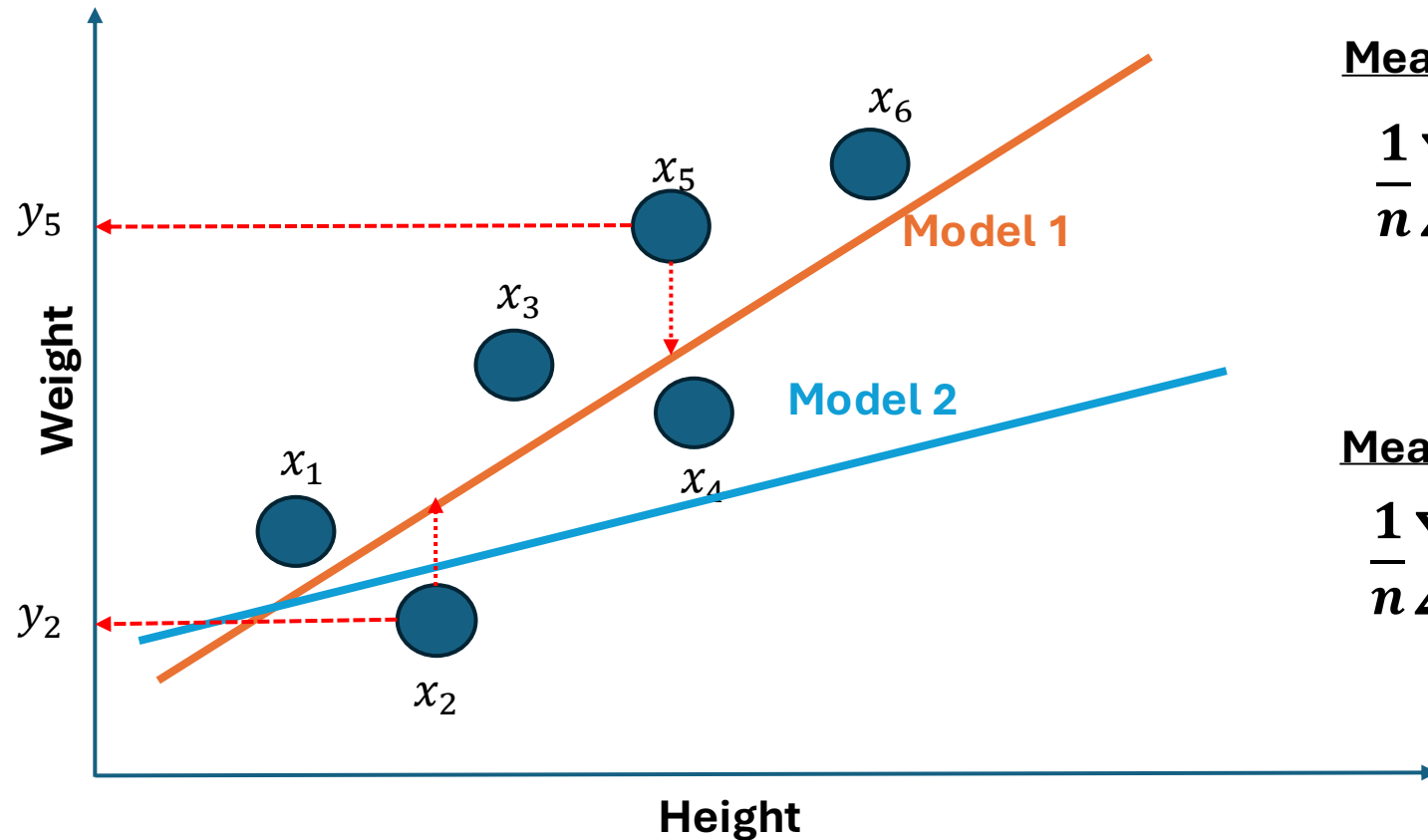
Lincoln Classifier



Lincoln Classifier



Lincoln Classifier



Mean-Squared Error (MSE)

$$\frac{1}{n} \sum (y_i - f(x_i))^2$$

Mean-Absolute Error (MSE)

$$\frac{1}{n} \sum |y_i - f(x_i)|$$

Loss / Cost Function

Binary Cross-Entropy

$$L = \frac{1}{n} \sum y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

Cross-Entropy

$$L = \frac{1}{n} \sum y_i \log(p_i)$$

P for the true label for sample i

1 for true label for sample i

Mean-Squared Error (MSE)

$$L = \frac{1}{n} \sum (y_i - f(x_i))^2$$

Mean-Absolute Error (MSE)

$$L = \frac{1}{n} \sum |y_i - f(x_i)|$$

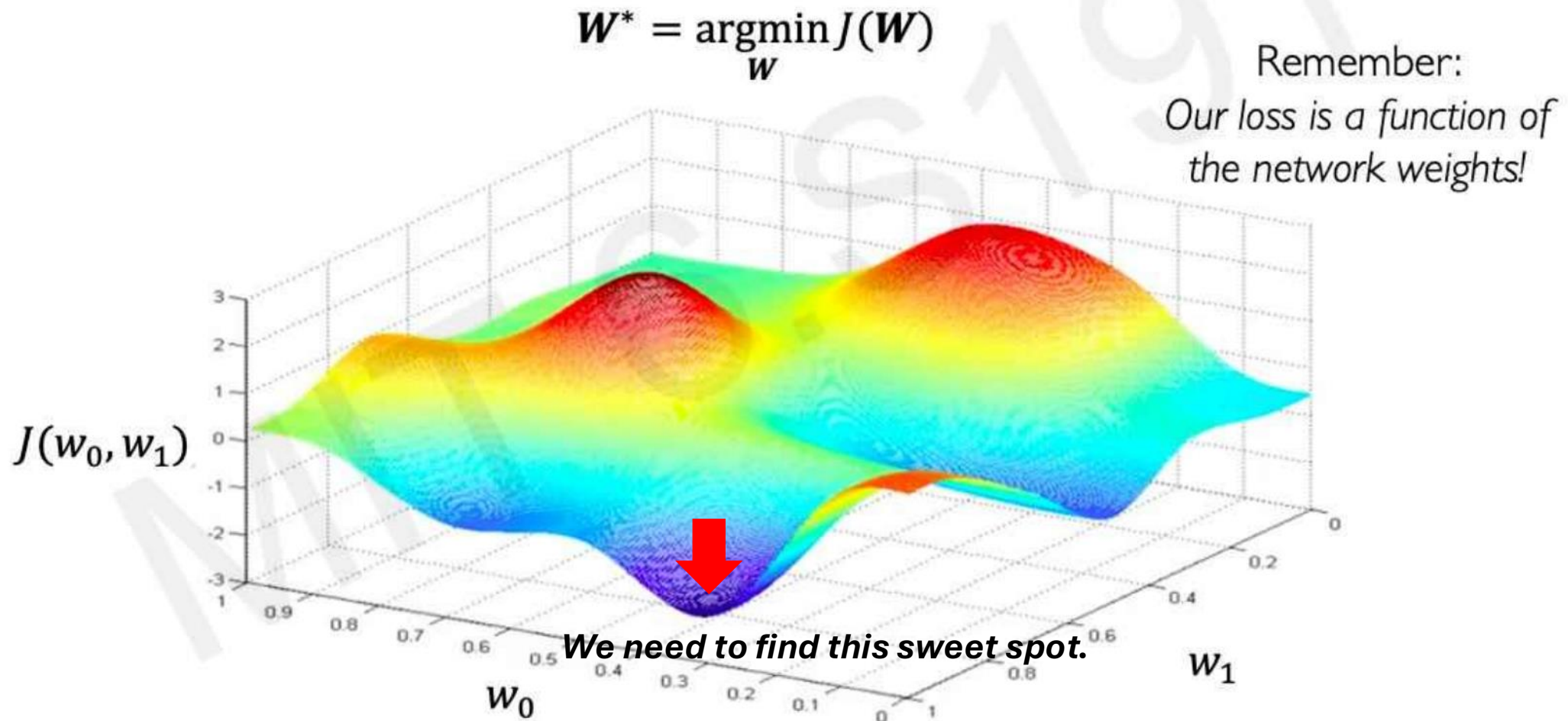


Model Training as Loss Minimization

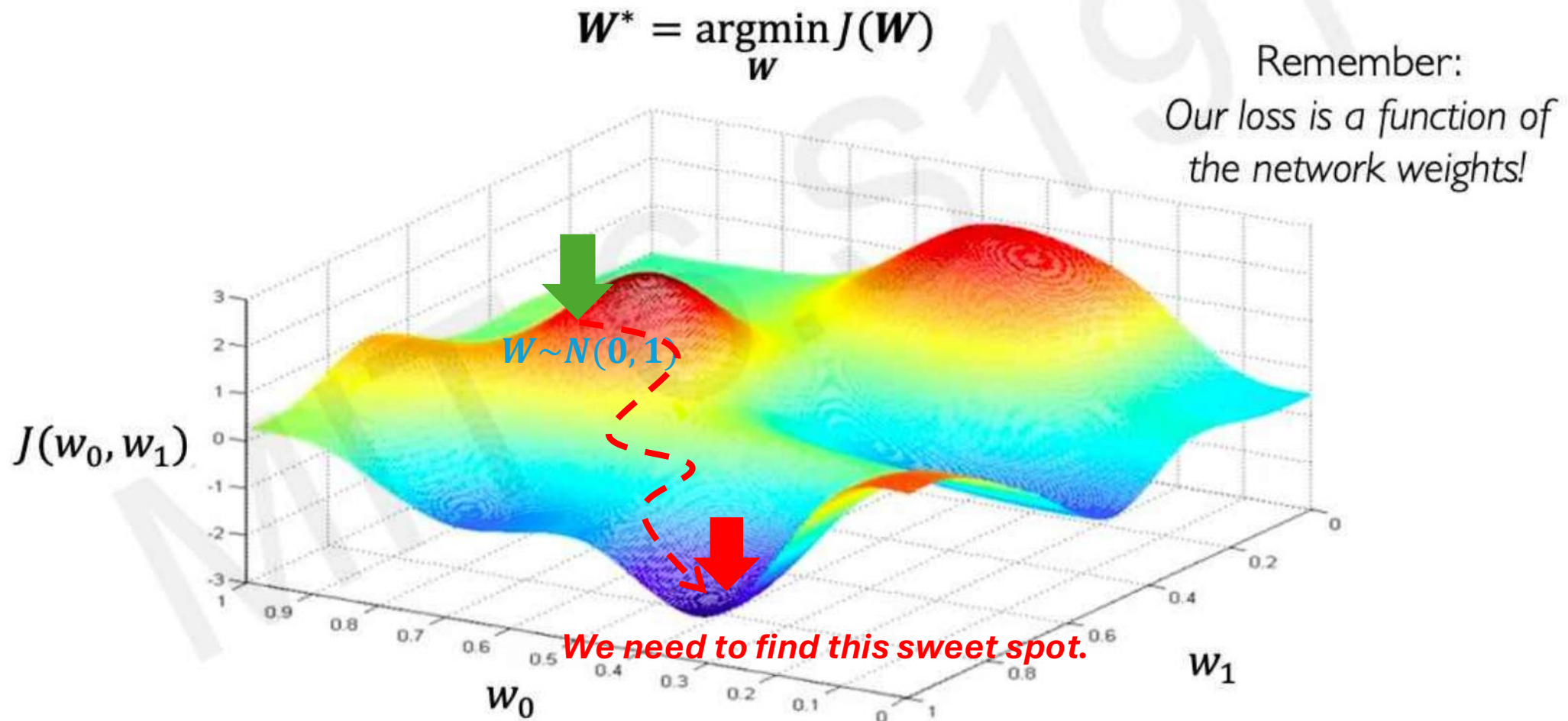
$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \operatorname{argmin}_W J(W)$$

Loss Surface – Non-Convex



Loss Surface – Non-Convex

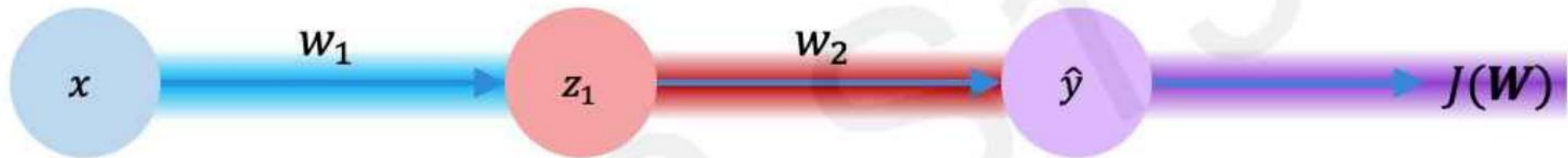


Gradient Descent

Algorithm

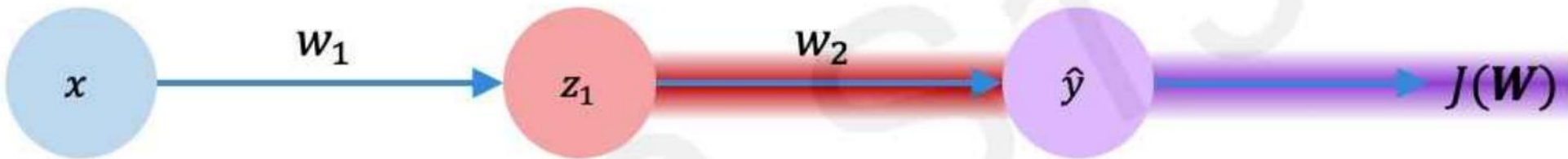
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence: *n_{epoch}*
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$ *Backpropagation*
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \boxed{\eta} \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$ *Update or Step learning rate*
5. Return weights *Model Weights*

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Computing Gradients: Backpropagation

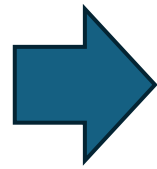


$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$



Linear Regression as a Special Case

$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$
$$W^* = \operatorname{argmin}_W J(W)$$



$$\frac{\partial J(W)}{\partial W} = 0$$

Solve for W ?

$$W = (X^T X)^{-1} X^T y \quad \text{Closed-form Solution}$$

Closed-form Solution vs Non-closed-form Solution

$$2x + 3 = 7$$



$$x = \frac{7 - 3}{2} = 2$$

$$x = \cos(x)$$



**There is no simple
algebraic expression
for x .**



**Gradient Descent
(iterative, approximate)**

Training Neural Networks is Difficult

- Hyper-parameters

- Optimizer-related

- Learning Rate
 - Epochs
 - Etc.

Training stability

- Model-related

- n layers
 - number of neurons
 - loss functions
 - Etc.

Model Generalization

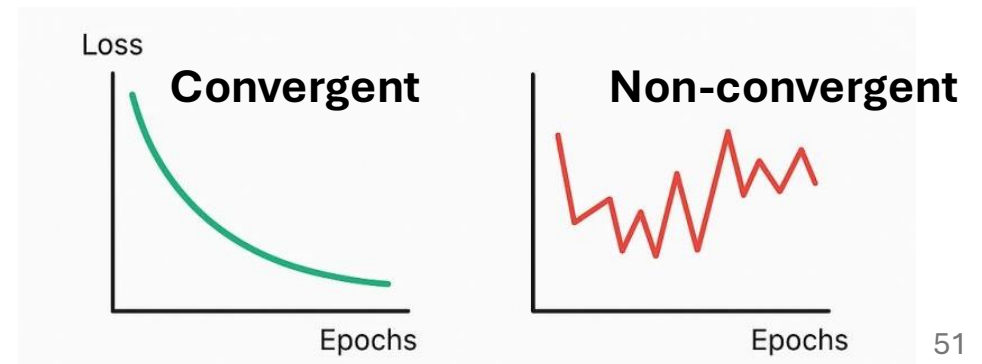
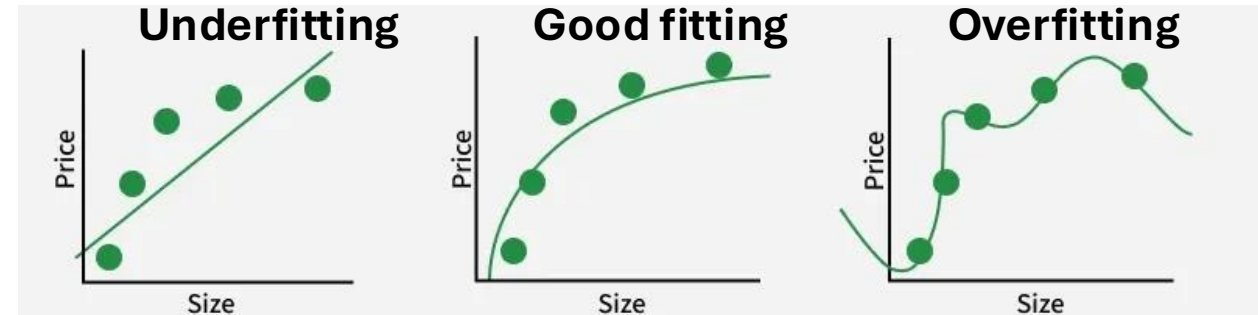
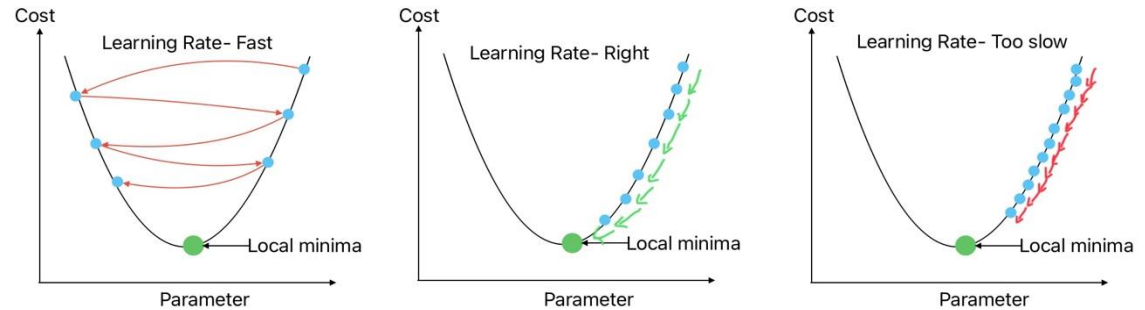
- Pre-processing-related

- Standardization
 - Min-max
 - Etc.

- Others

Model Convergence

Impact of Step Size or learning rate on Gradient Decent Performance



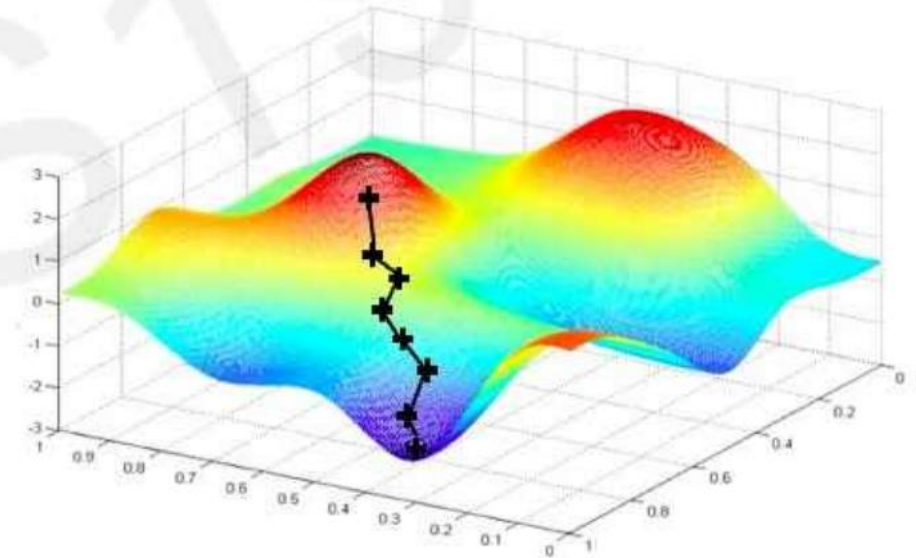
Adaptive Learning Rate

Algorithm	TF Implementation	Torch Implementation	Reference
• SGD	 <code>tf.keras.optimizers.SGD</code>	 <code>torch.optim.SGD</code>	Kiefer & Wolfowitz, 1952.
• Adam	 <code>tf.keras.optimizers.Adam</code>	 <code>torch.optim.Adam</code>	Kingma et al., 2014.
• Adadelta	 <code>tf.keras.optimizers.Adadelta</code>	 <code>torch.optim.Adadelta</code>	Zeiler et al., 2012.
• Adagrad	 <code>tf.keras.optimizers.Adagrad</code>	 <code>torch.optim.Adagrad</code>	Duchi et al., 2011.
• RMSProp	 <code>tf.keras.optimizers.RMSProp</code>	 <code>torch.optim.RMSProp</code>	

Vanilla Gradient Descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$
4. Update weights, $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$
5. Return weights



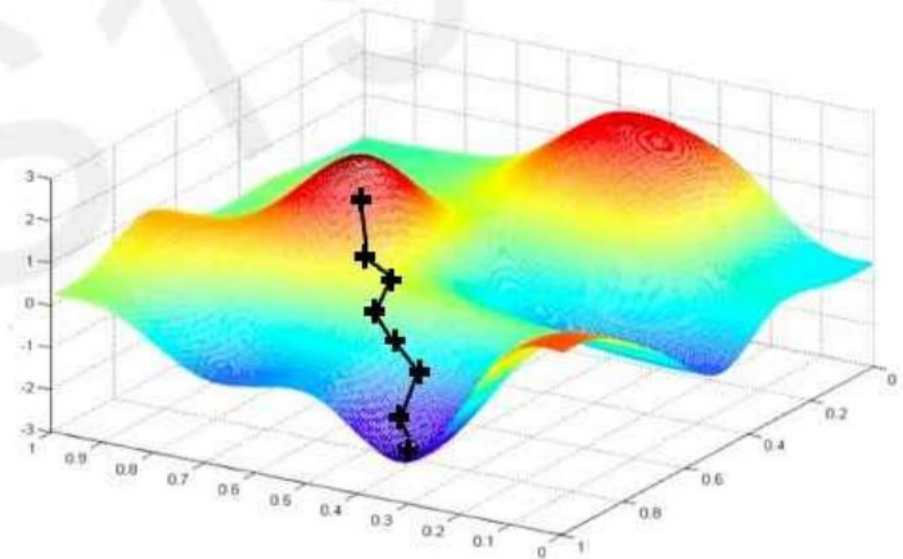
$$J(\mathbf{w}) = \sum_{i=1}^n L_i \quad \Rightarrow \quad O(n) \text{ complexity}$$

What if 1M samples ?

Stochastic Gradient Descent

Algorithm

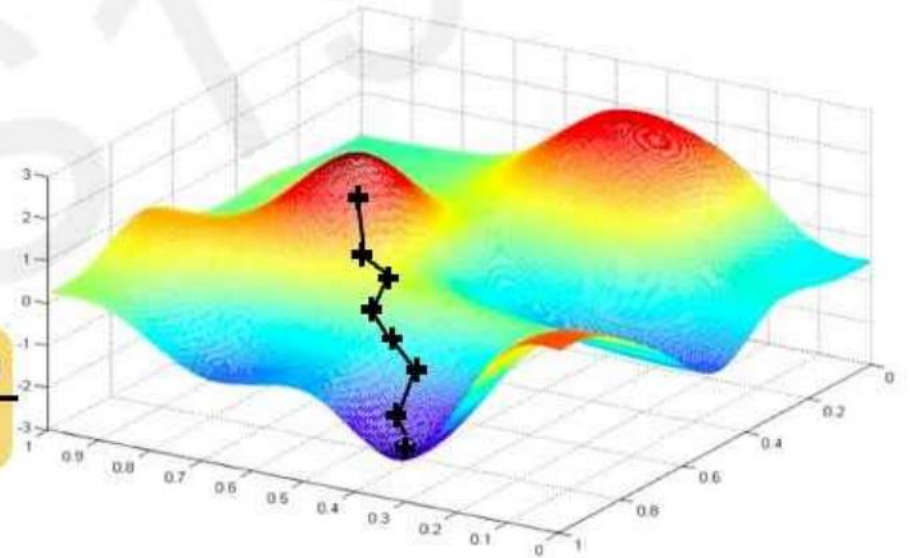
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



Stochastic Gradient Descent

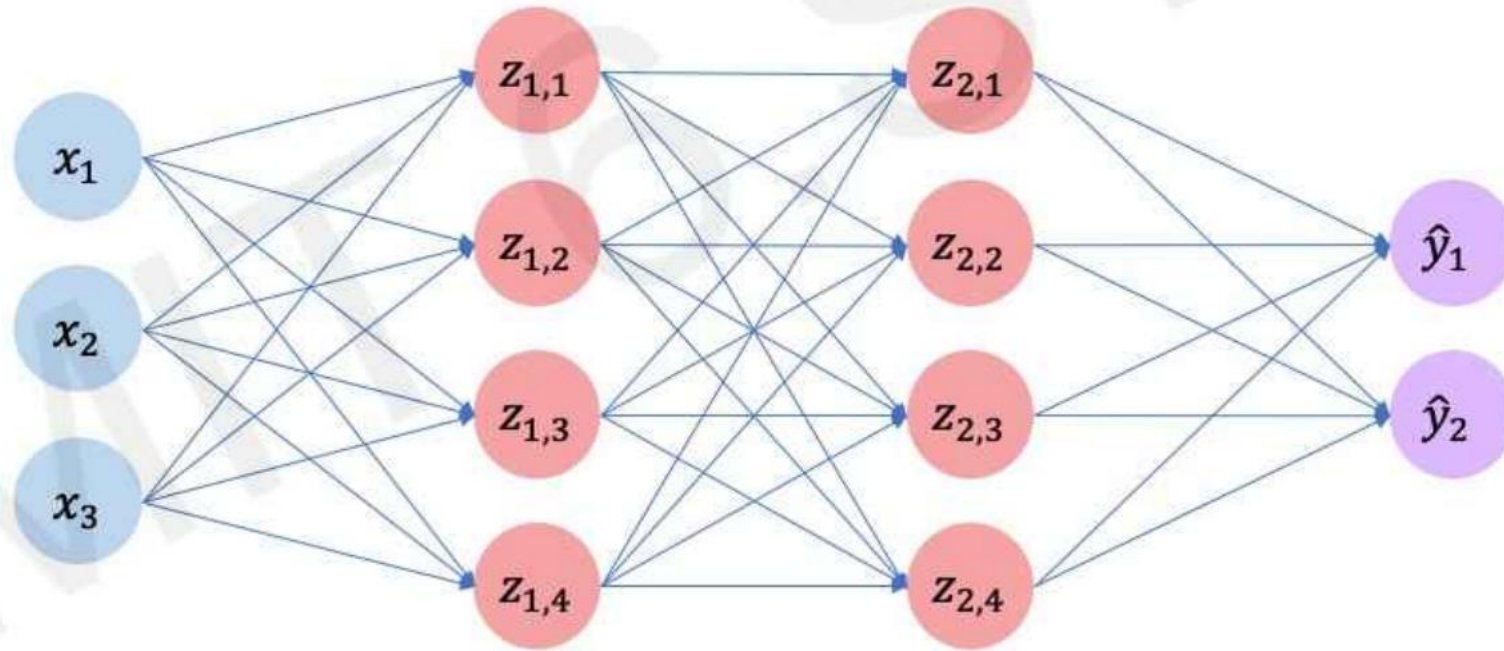
Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



Fast to compute and a much better estimate of the true gradient!

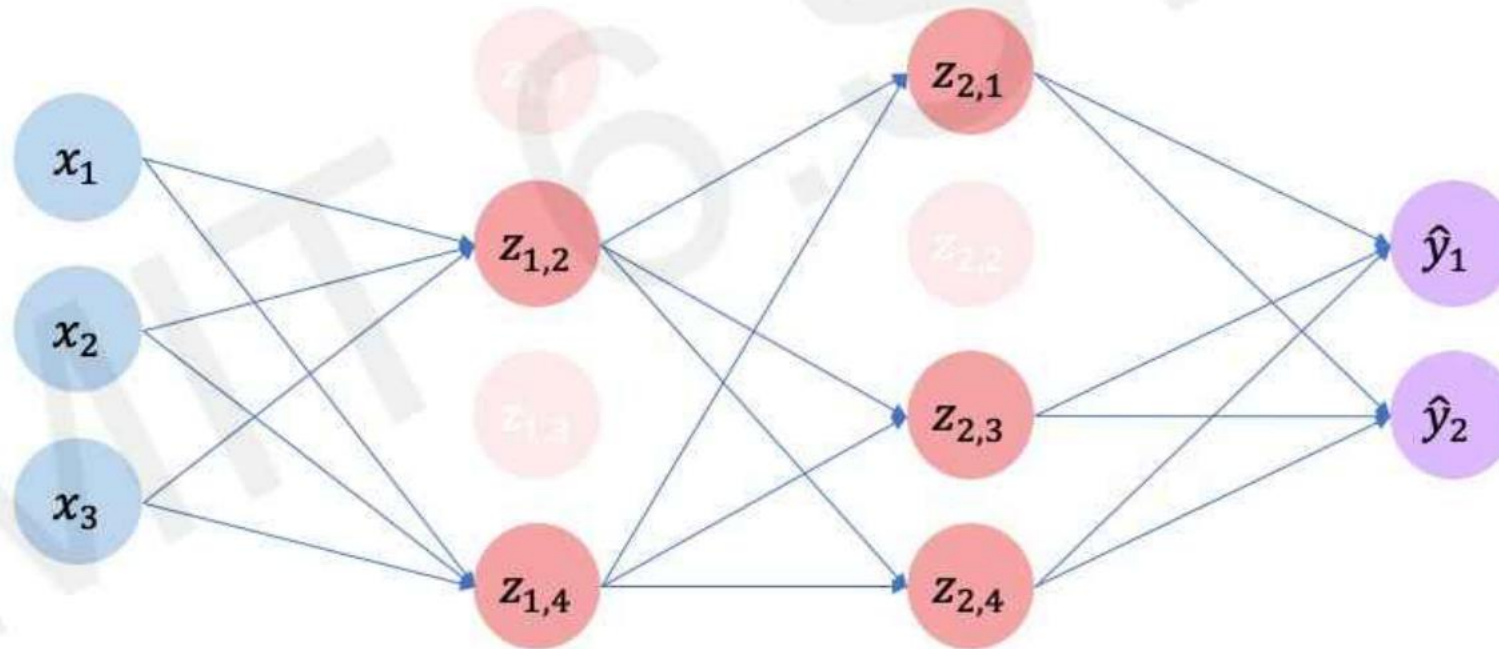
Regularization



Regularization - Dropout

- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node

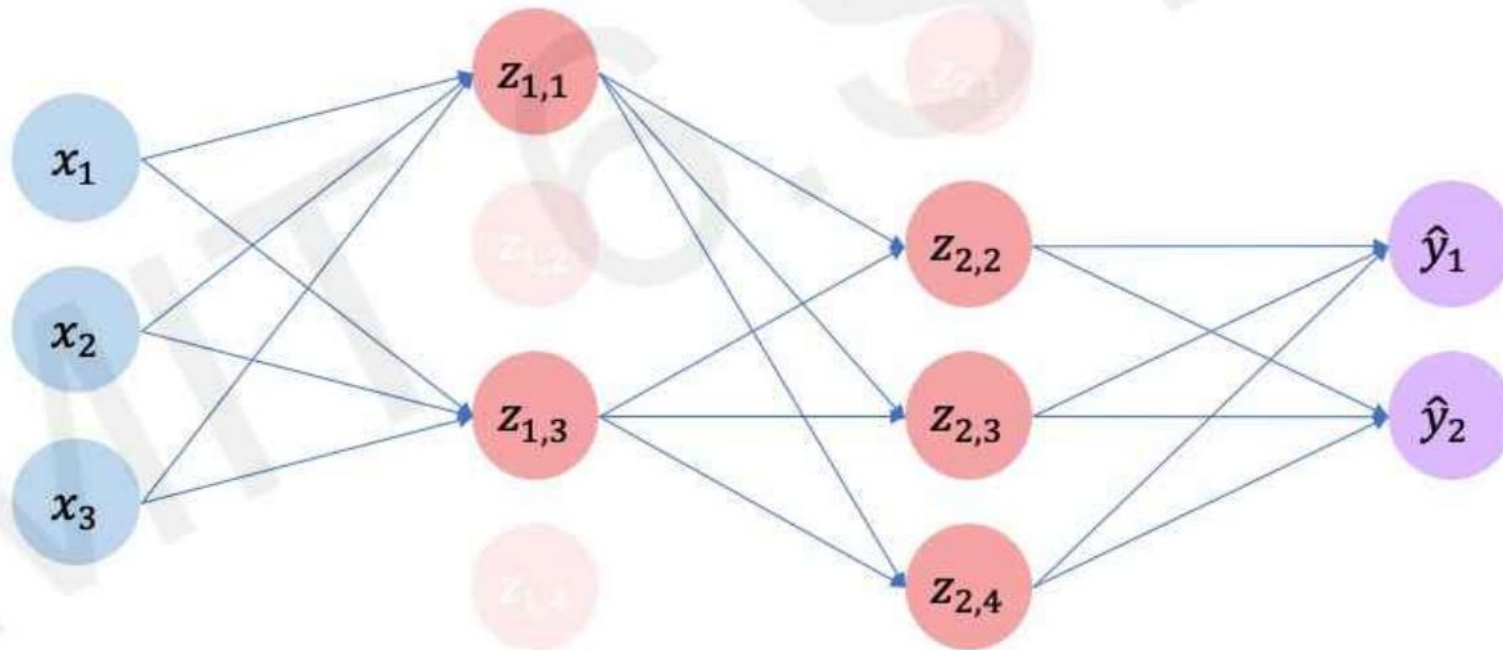
```
tf.keras.layers.Dropout(p=0.5)  
torch.nn.Dropout(p=0.5)
```



Regularization - Dropout

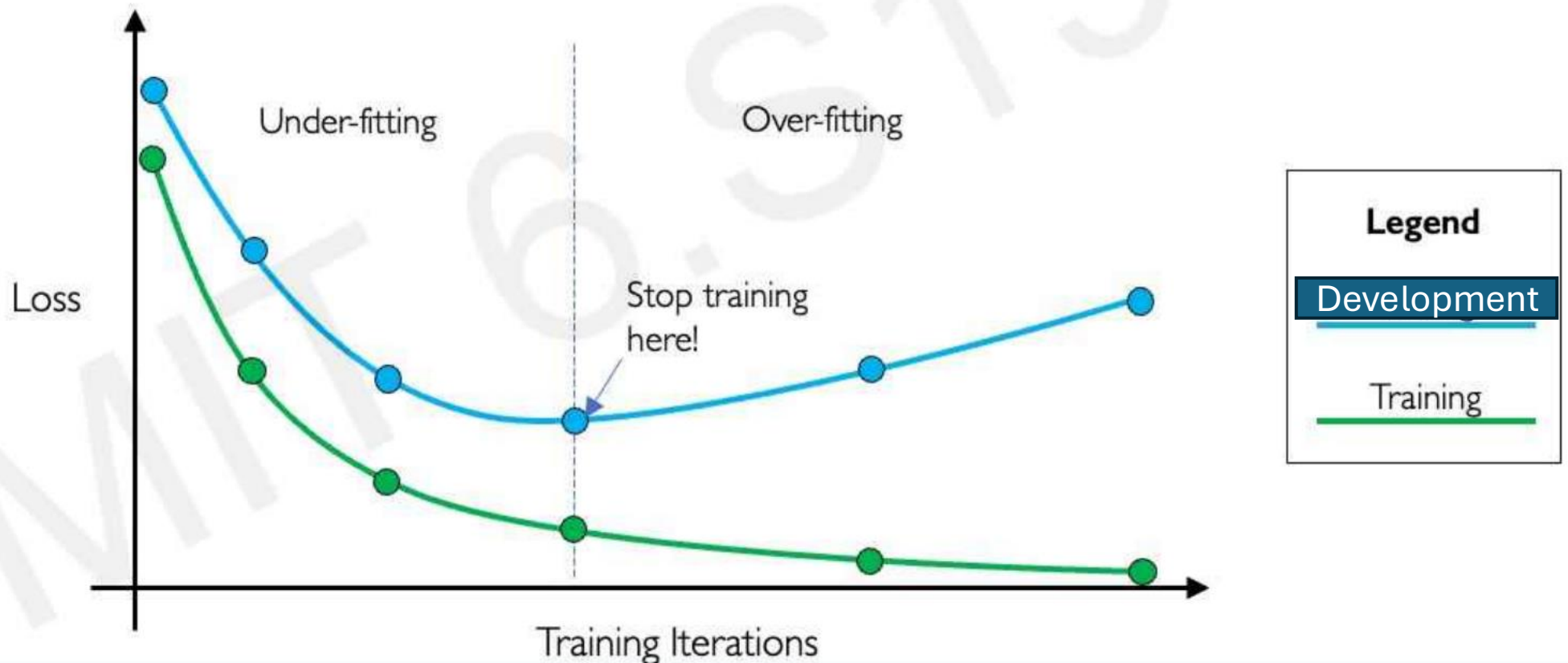
- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node

```
tf.keras.layers.Dropout(p=0.5)  
torch.nn.Dropout(p=0.5)
```



Early Stopping

- Stop training before we have a chance to overfit



Regularization

L1 Regularization

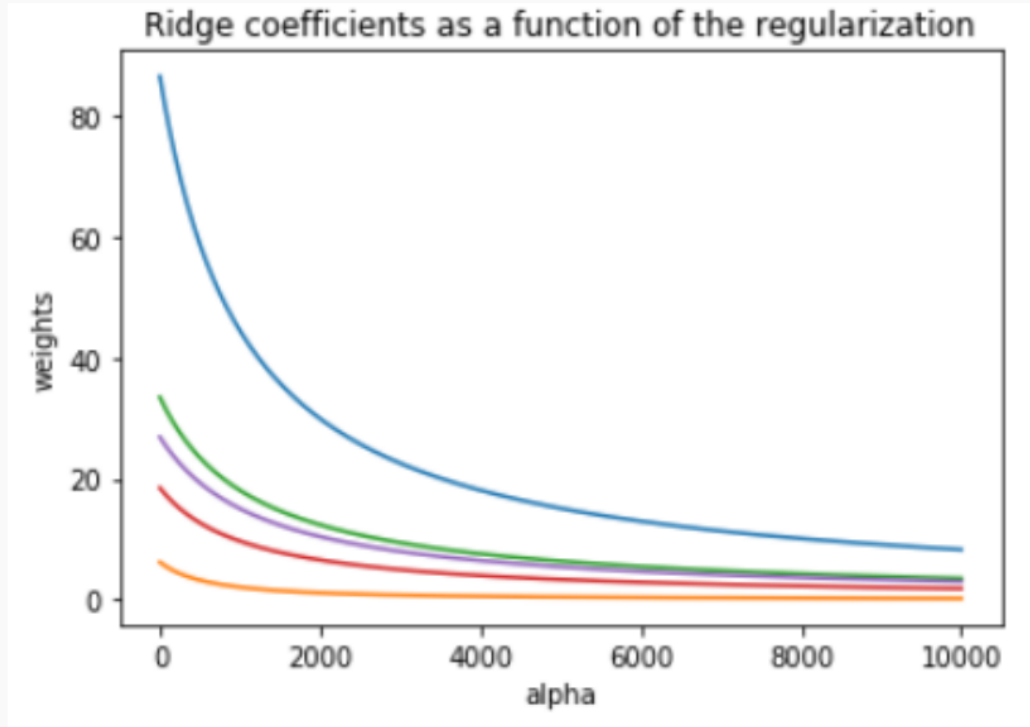
$$\begin{array}{l} \text{Modified loss} \\ \text{function} \end{array} = \text{Loss function} + \boxed{\lambda} \sum_{i=1}^n |W_i|$$

***Regularization Factor
(sometimes, α)***

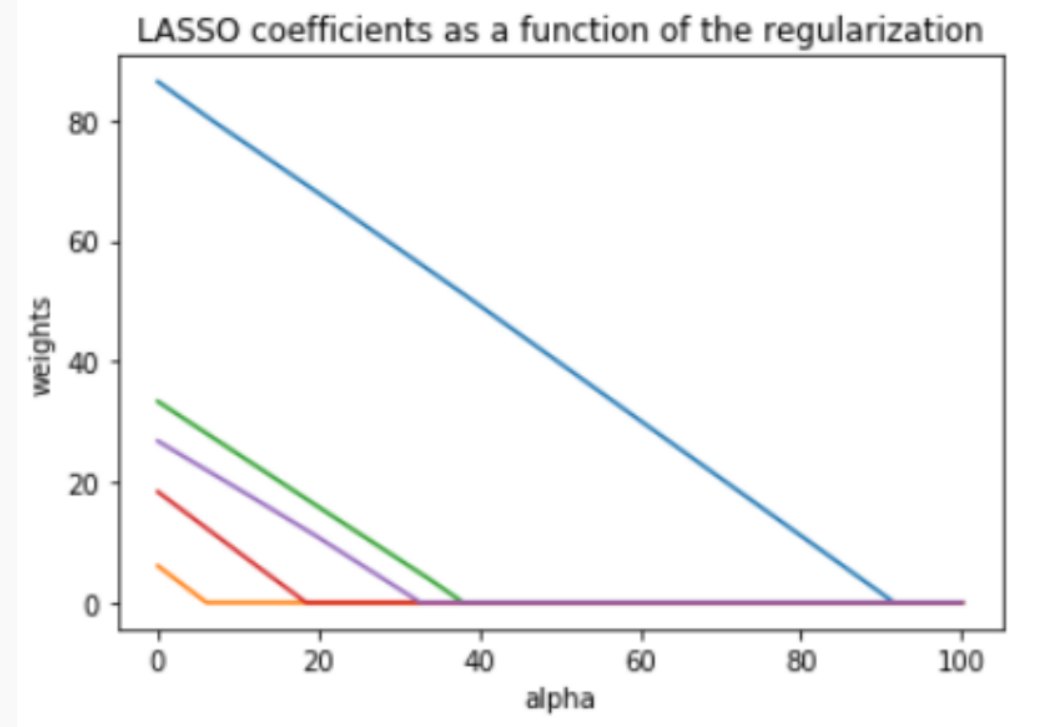
L2 Regularization

$$\begin{array}{l} \text{Modified loss} \\ \text{function} \end{array} = \text{Loss function} + \boxed{\lambda} \sum_{i=1}^n W_i^2$$

Linear Regression as a Special Case – Ridge vs Lasso

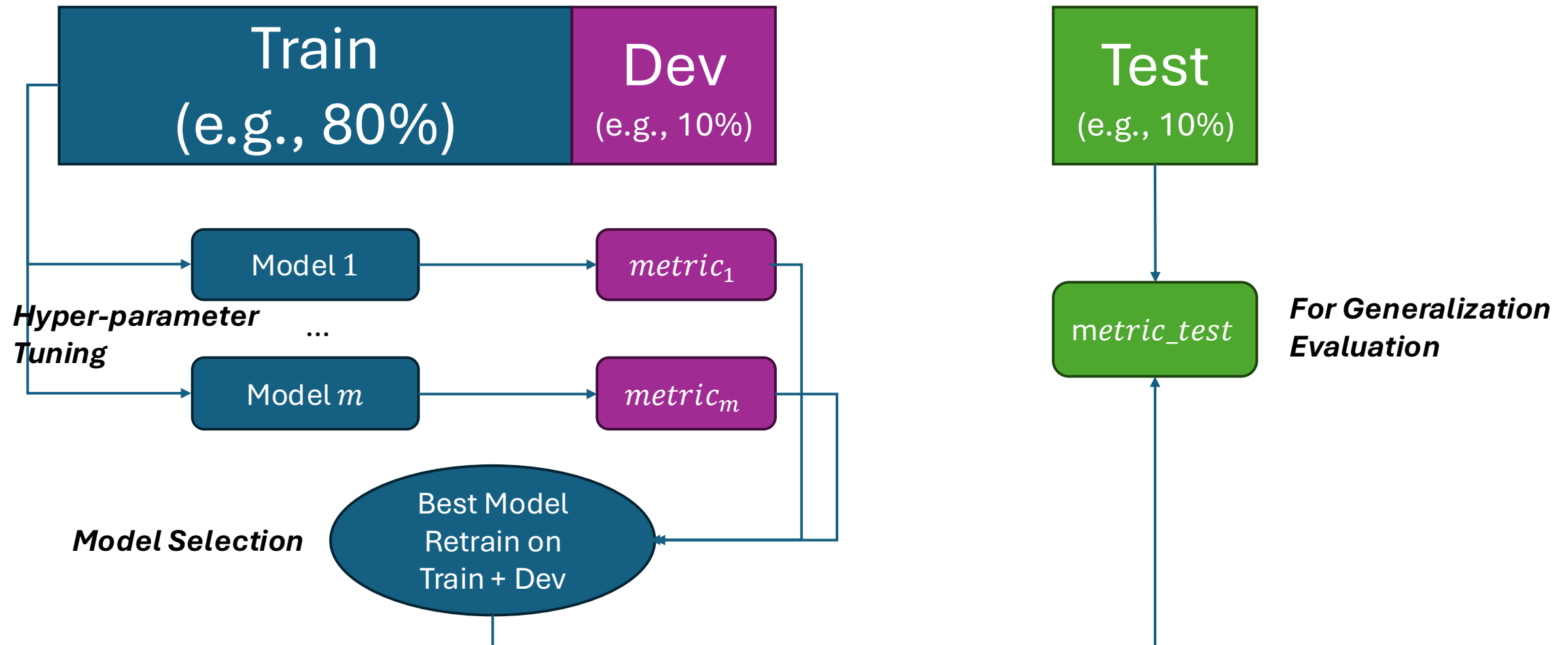


The values of the coefficients decrease as lambda increases, but they are not nullified.

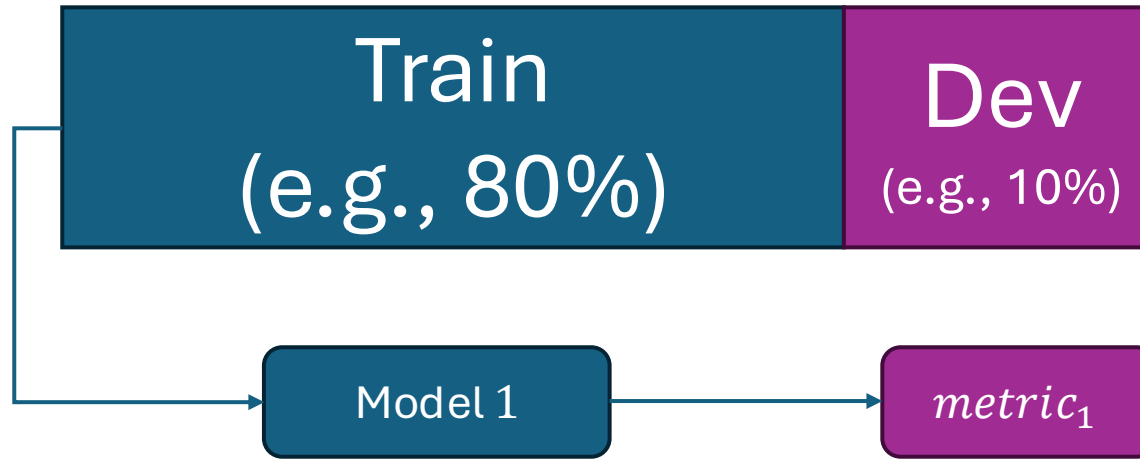


The values of the coefficients decrease as lambda increases, and are nullified fast.

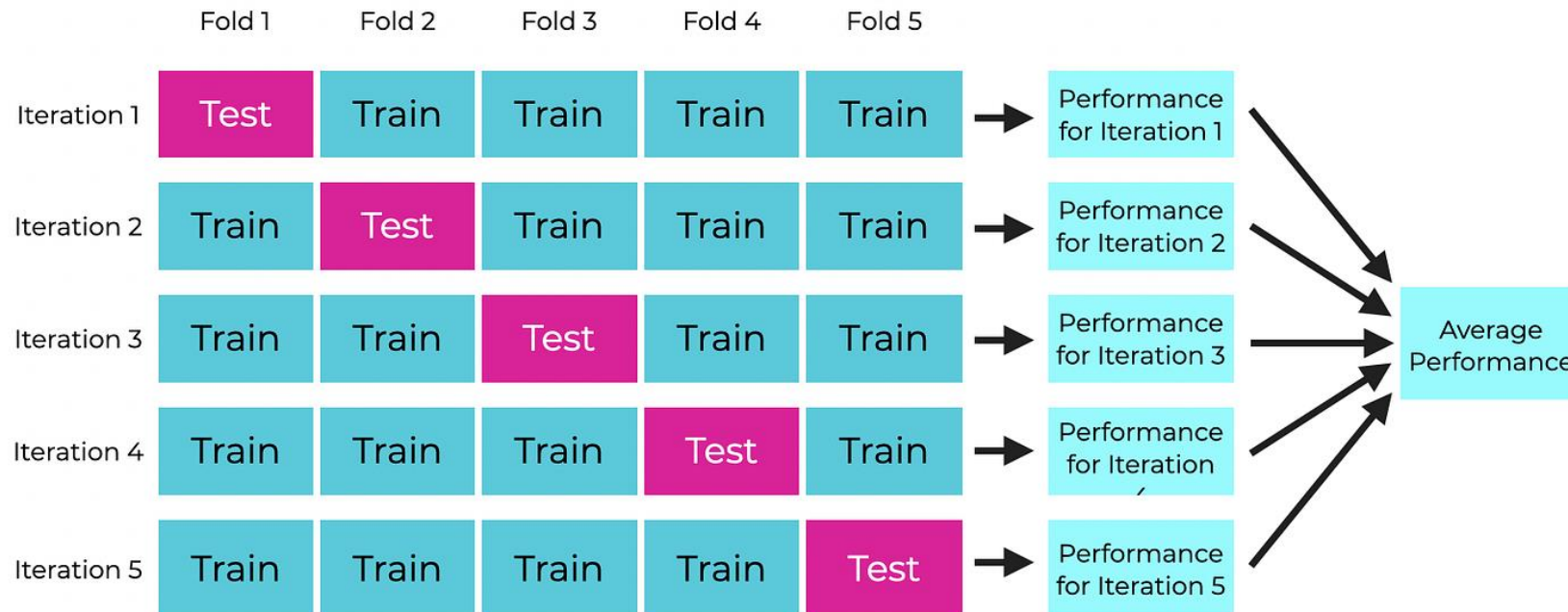
Train-Development/Validation-Test Split



K-Fold Cross-Validation



Common for
Deep Learning



Common for
Machine
Learning

Recap - Loss / Cost Function

Binary Cross-Entropy

$$L = \frac{1}{n} \sum y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

Cross-Entropy

$$L = \frac{1}{n} \sum y_i \log(p_i)$$

P for the true label for sample i

1 for true label for sample i

Mean-Squared Error (MSE)

$$L = \frac{1}{n} \sum (y_i - f(x_i))^2$$

Mean-Absolute Error (MSE)

$$L = \frac{1}{n} \sum |y_i - f(x_i)|$$

These are just a proxy to a human-readable performance.

Model Training as Loss Minimization

$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \operatorname{argmin}_W J(W)$$

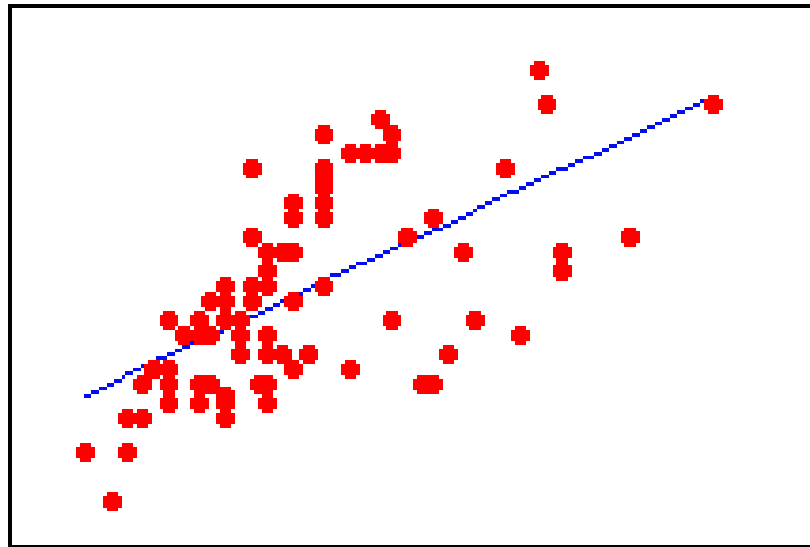
R-Squared for Regression Problems – [0,1]

It essentially acts as a baseline evaluation metric to score how well your predictive model fits your observed data.

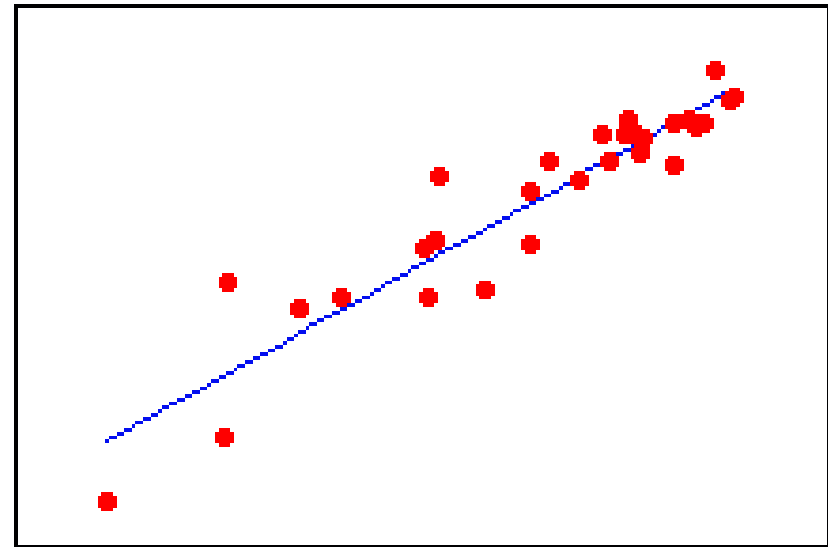
$$R^2 = 1 - \frac{\sum_i^N (y_i - \hat{y}_i)^2}{\sum_i^N (y_i - \bar{y})^2}$$

Plots of Observed Responses Versus Fitted Responses for Two Regression Models

Fitted
responses



Observed responses



Observed responses

Confusion matrix for Binary Classification

When a classification algorithm (like logistic regression) is used, the results can be summarize in a ($k \times k$) table as such:

	Predicted no AHD ($\hat{Y} = 0$)	Predicted AHD ($\hat{Y} = 1$)
Truly no AHD ($Y = 0$)	110	54
Truly AHD ($Y = 1$)	53	86

The table above was a classification based on a logistic regression model to predict AHD based on “3” predictors: X_1 = Age, X_2 = Sex, and X_3 = interaction between Age and Sex.

What are the false positive and false negative rates for this classifier?

Precision, Recall, and F1-Score

Key: **tp** = True Positive, **tn** = True Negative, **fp** = False Positive, **fn** = False Negative

Metric Name	Metric Formula	Code	When to use
Accuracy	$\text{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$	<code>tf.keras.metrics.Accuracy()</code> or <code>sklearn.metrics.accuracy_score()</code>	Default metric for classification problems. Not the best for imbalanced classes.
Precision	$\text{Precision} = \frac{tp}{tp + fp}$	<code>tf.keras.metrics.Precision()</code> or <code>sklearn.metrics.precision_score()</code>	Higher precision leads to less false positives.
Recall	$\text{Recall} = \frac{tp}{tp + fn}$	<code>tf.keras.metrics.Recall()</code> or <code>sklearn.metrics.recall_score()</code>	Higher recall leads to less false negatives.
F1-score	$\text{F1-score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$	<code>sklearn.metrics.f1_score()</code>	Combination of precision and recall, usually a good overall metric for a classification model.
Confusion matrix	NA	Custom function or <code>sklearn.metrics.confusion_matrix()</code>	When comparing predictions to truth labels to see where model gets confused. Can be hard to use with large numbers of classes.

Other Confusion tables

Based on $\pi = 0.4$:

	Predicted no AHD ($\hat{Y} = 0$)	Predicted AHD ($\hat{Y} = 1$)
Truly no AHD ($Y = 0$)	93	71
Truly AHD ($Y = 1$)	38	101

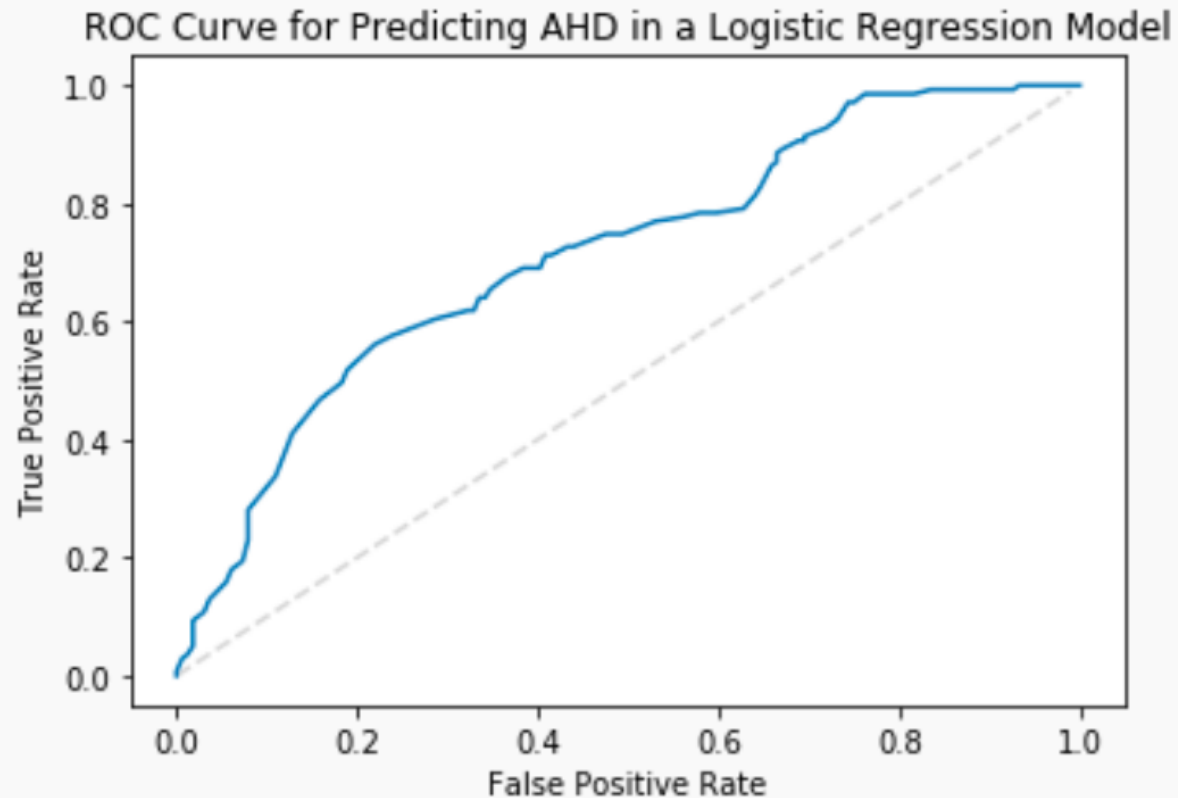
What has improved? What has worsened?

Based on $\pi = 0.6$:

	Predicted no AHD ($\hat{Y} = 0$)	Predicted AHD ($\hat{Y} = 1$)
Truly no AHD ($Y = 0$)	138	26
Truly AHD ($Y = 1$)	74	65

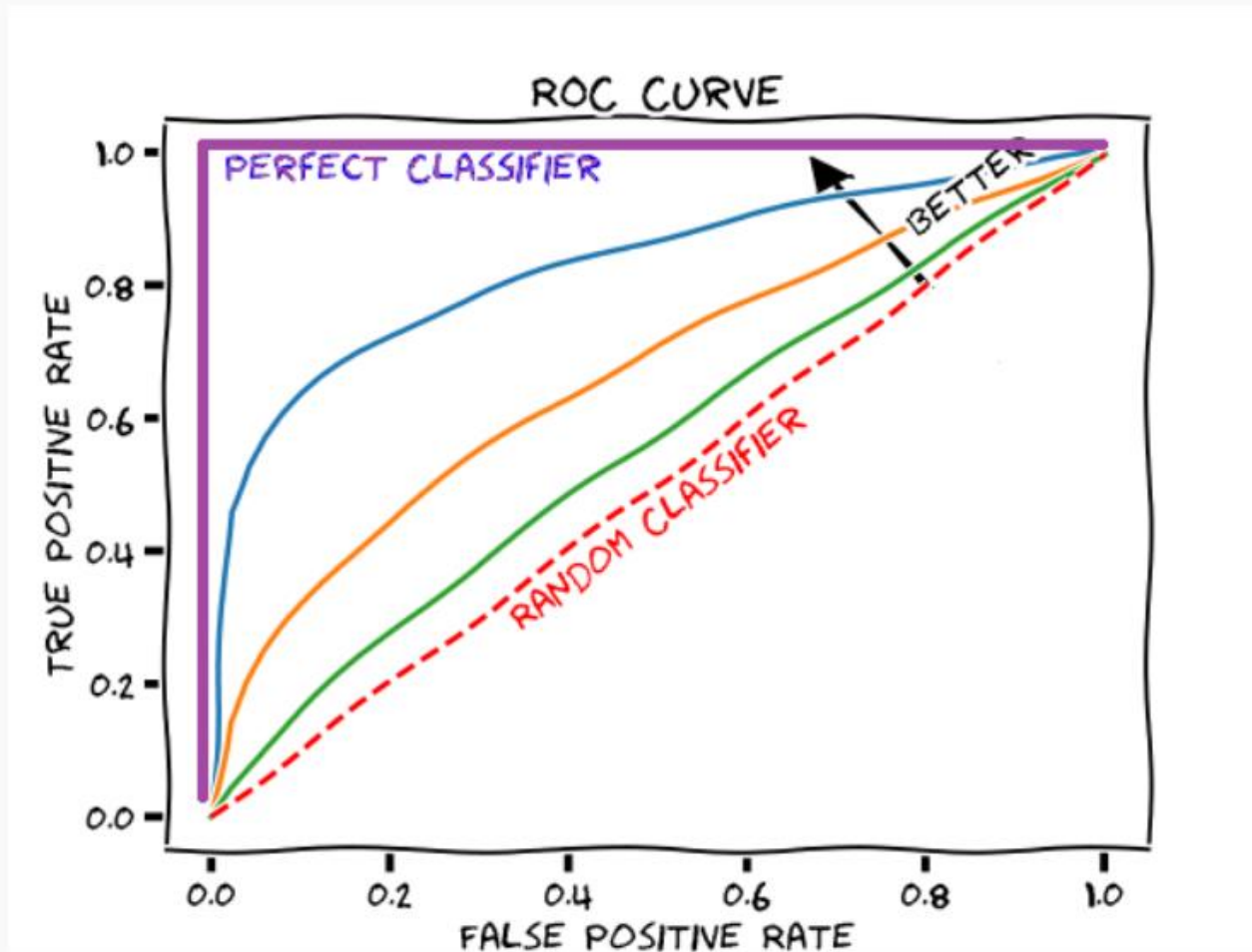
Which should we choose? Why?

ROC Curves and Area-Under-the-Curve



What is the shape of an ideal ROC curve?

ROC Curve Example



AUC for measuring classifier performance

The overall performance of a classifier, calculated over all possible thresholds, is given by the **area under the ROC curve** (AUC).

An ideal ROC curve will hug the top left corner, so the larger the AUC the better the classifier.

This AUC then can be use to compare various approaches to classification: Logistic regression, k -NN, Decision Trees (to come), etc.